

基于智能优化算法的无人机烟幕干扰策略建模与优化

摘要

烟幕干扰弹作为一种成本低、效费比高的防御手段，主要借助化学燃烧或爆炸形成烟幕或气溶胶云团，在目标前方特定空域构建遮蔽屏障。本文基于三维运动学分析与约束优化理论，系统研究不同战术背景下的烟幕干扰弹最优投放策略。

针对问题一，在给定无人机以120m/s速度等高飞行，受领任务1.5s后投弹，3.6s后起爆的条件下，通过建立无人机匀速直线运动与烟幕干扰弹平抛运动复合模型，结合线段与球面相交的有效遮蔽判定准则，计算出烟幕对导弹的有效遮蔽时长为1.392s。

针对问题二，将无人机速度、投放延迟、起爆延迟、飞行角度作为决策变量，构建以有效遮蔽时间最大化为目标的优化模型，通过差分进化算法搜索最优解。采用自适应时间步长近目标0.005s、远目标0.02s计算遮蔽时间，结合约束惩罚函数处理物理合理性限制。优化结果显示，最优遮蔽时长为4.82s，对应飞行角度52.95°、无人机速度119.31m/s、投放延迟0.1110s、起爆延迟1.5209s。

针对问题三，以3枚烟幕干扰弹对M1的有效遮蔽总时间最大化为目标，进行多参数优化。采用增强遗传算法开展优化：先通过智能策略生成初始种群，结合约束惩罚函数计算适应度，采用锦标赛选择、智能交叉、自适应变异生成新种群，通过分层扫描确定有效遮蔽时间，经多代迭代筛选出最优方案。优化结果显示，无人机飞行角度179.5°，飞行速度112.5m/s，总有效遮蔽时间6.600s，具体结果保存至result1.xlsx。

针对问题四，以3架无人机各投1枚烟幕弹对M1的有效遮蔽总时间最大化为目标，通过多机协同模型模拟烟幕弹投放与遮蔽过程，采用分层扫描策略追踪导弹与多烟幕云团的位置变化，同时开展多参数优化，通过智能策略生成初始种群，结合多层惩罚函数计算适应度，经多代迭代筛选出最优方案。优化结果显示，有效遮蔽总时间为11.7s，具体结果保存至result2.xlsx。

针对问题五，以三枚导弹的协同遮蔽效果最大化为核心目标，构建多无人机协同优化模型。将45维决策变量纳入扩展遗传算法框架，结合分层约束处理与自适应修复机制，生成兼顾时空覆盖与资源分配的最优方案。采用分导弹遮蔽分析与协同效率系数量化评估，逐时刻追踪多烟幕云团对导弹群的动态遮蔽过程，优化结果显示，三枚导弹的综合有效遮蔽总时长达21.2s，具体结果保存至result3.xlsx。

综上所述，本研究围绕无人机烟幕干扰策略优化问题，建立了从单弹到多弹、单机到多机的递进式研究体系。研究成果为无人机烟幕防御系统的策略制定提供了科学依据，对提升防空反导作战效能具有重要的理论价值与实践意义。

关键词：烟幕干扰 遮蔽策略 智能优化算法 多目标协同 遗传算法

一、问题重述

1.1. 问题背景

烟幕干扰弹借助化学燃烧或爆炸反应形成烟幕或气溶胶云团，能在目标前方特定空域构建遮蔽屏障干扰敌方导弹，其成本低廉且效费比突出。随着技术发展，目前已能通过多种方式实现烟幕干扰弹的定点精准抛撒，不仅能在抛撒前精准控制其抵达预定位置，还可利用时间引信对起爆时间进行调控。

1.2. 问题提出

现考虑无人机挂载烟幕干扰弹执行投放任务，无人机巡飞时受领任务后投放干扰弹，在保护目标和来袭武器间形成遮蔽，单架无人机投放两枚干扰弹的时间间隔不得少于1s。干扰弹脱离无人机后受重力作用运动，起爆后形成球状烟幕云团，该云团以3m/s匀速下沉，其中心10m范围内在起爆20s内可有效遮蔽目标。来袭的空地导弹飞行速度为300m/s，直指掩护真目标（半径7m，高10m）的假目标。以假目标为原点，水平面为xy平面，真目标下底面的圆心为(0, 200, 0)。雷达发现导弹时，导弹M1、M2、M3分别位于(20000,0,2000)、(19000,600,2100)、(18000,-600,1900)；无人机FY1至FY5位置分别为(17800,0,1800)、(12000,1400,1400)、(6000,-3000,700)、(11000,2000,1800)、(13000,-2000,1300)。控制中心发现目标后立即指派无人机投放烟幕干扰弹避免导弹发现真目标，无人机可瞬时调整方向，随后以70-140m/s等高度匀速直线飞行，各无人机航向和速度可不相同且确定后不再调整。

为实现有效干扰，需设计包括干扰弹投放位置及起爆点、无人机飞行方向、速度等的投放策略，使干扰弹对真目标的有效遮蔽时间尽可能长（多枚干扰弹的遮蔽可不连续）。要求建立数学模型解决下列问题：

问题一：调用无人机FY1对M1投放1枚烟幕干扰弹实施干扰。已知FY1以120m/s的速度朝假目标飞行，受领任务1.5s后投放该干扰弹，投放后间隔3.6s起爆，需计算烟幕干扰弹对M1的有效遮蔽时长。

问题二：调用无人机FY1对M1投放1枚烟幕干扰弹实施干扰，需确定FY1的干扰弹的投放位置和起爆点，以及飞行方向、速度，以实现遮蔽时间的最大化。

问题三：调用无人机FY1对M1投放3枚烟幕干扰弹实施干扰，需制定具体投放策略，并将结果保存至文件result1.xlsx。

问题四：调用FY1、FY2、FY3三架无人机各向M1投放1枚烟幕干扰弹实施干扰，需给出相应投放策略，并将结果保存至文件result2.xlsx。

问题五：调用5架无人机（每架最多投放3枚烟幕干扰弹）对M1、M2、M3三枚导弹进行干扰，需设计投放策略，并将结果保存到文件result3.xlsx。

二、问题分析

2.1. 问题一的分析

问题一聚焦于无人机FY1投放1枚烟幕干扰弹对M1的干扰效果，核心是计算有效遮蔽时长。首先根据无人机固定的飞行速度和方向，结合投放延迟确定干扰弹投放位置；其次依据干扰弹的水平惯性与重力作用，结合起爆延迟计算起爆点位置，烟幕干扰弹起爆后形成云团并匀速下沉[1]，需实时分析云团与导弹的相对位置；最后通过生成真目标表面关键采样点，判断导弹到各采样点的视线线段是否被烟幕云团遮挡，累计所有采样点均被遮挡时的总时长，即为有效遮蔽时长。

2.2. 问题二的分析

问题二旨在通过优化无人机FY1的飞行参数及干扰弹投放[2]、起爆点，使M1被干扰的有效遮蔽时间尽可能长。首先，确定无人机飞行方向、飞行速度、投放延迟和起爆延迟为决策变量，根据飞行方向计算水平方向向量，结合速度和投放延迟确定投放位置，依据起爆延迟计算干扰弹下落位移得到起爆点；其次，模拟烟幕下沉和导弹飞行轨迹，采用自适应时间步长追踪位置，通过判断真目标所有采样点视线是否被遮挡计算遮蔽时间；最后，构建以遮蔽时间为目标的函数，引入参数约束和物理合理性惩罚，利用差分进化算法搜索最优参数组合，得到有效遮蔽时间最长的投弹策略。

2.3. 问题三的分析

问题三聚焦于无人机FY1投放3枚烟幕干扰弹对M1的协同干扰效果，核心是通过参数优化得到有效遮蔽总时间（重叠时间只计一次）。首先，确定优化变量包括无人机飞行角度、飞行速度，以及3枚干扰弹的投放延迟和起爆延迟；其次，根据飞行角度与速度计算无人机运动轨迹，结合各弹投放延迟确定对应投放点，依据起爆延迟计算每枚干扰弹的下落位移得到起爆位置；随后，逐时刻追踪各烟幕云团与导弹的实时位置，通过判断真目标采样点视线是否被任一烟幕遮挡，累计非重叠时段的总时长；最后，构建以有效遮蔽总时间为目标的优化模型，引入物理约束与惩罚机制，采用改进差分进化算法搜索最优参数组合，得到最长时间有效遮蔽策略整理保存至result1.xlsx。

2.4. 问题四的分析

问题四围绕三架无人机FY1、FY2、FY3各投放1枚烟幕干扰弹对M1的协同干扰效果，通过参数优化得到有效遮蔽总时间。首先分别为三架无人机规划飞行方向与速度，确定各自的飞行轨迹及干扰弹投放点；其次根据各无人机的飞行角度计算水平方向向量，结合速度和投放延迟确定各自的投放点；随后分析三枚烟幕云团的下沉过程，结合其有效遮蔽范围规划各云团对真目标的遮蔽时段，实现多云团遮蔽效果的互补与衔接；最后关联M1的飞行路径，通过优化三架无人机的参数及干扰弹投放、起爆参数，使有效遮蔽总时间最大化，并将投放策略整理保存至result2.xlsx。

2.5. 问题五的分析

问题五针对多导弹威胁构建多无人机协同干扰模型，需调度5架无人机（每架至多投放3枚）对M1、M2、M3实施干扰。首先综合考虑三枚导弹的飞行速度、初始位置及路径差异，为每架无人机分配目标导弹及干扰弹数量；其次针对各无人机，规划其飞行方向与速度，确定各枚干扰弹的投放时序、投放点及起爆点；随后分析对导弹的有效遮蔽时段，协调多机多弹的遮蔽效果，避免资源浪费或覆盖重叠不足；最后通过建立多目标优化模型，在满足约束条件前提下使三枚导弹干扰真目标的有效遮蔽总时间最长，并将有效策略整理保存至result3.xlsx。

三、模型假设

1. 烟幕干扰弹脱离无人机后以无人机飞行速度为水平初速度，在重力作用下运动，起爆后瞬时形成球状烟幕云团。
2. 导弹在模型中当做质点处理，不考虑空气阻力，风速等外力影响。
3. 遮蔽有效性判断标准为：从导弹当前位置到真目标表面关键点的视线被烟幕云团球体遮挡，则认为实现有效遮蔽，不考虑烟幕对不同波长光线的吸收与散射差异。

四、符号说明

符号	定义	单位
v_d	导弹飞行速度	m/s
v_w	无人机飞行速度	m/s
r_{yx}	烟幕有效遮蔽半径	m
t_{yx}	烟幕有效遮蔽持续时间	s
t_{tf}	投放延迟时间	s
t_{qb}	起爆延迟时间	s
Y_{tf}	烟幕投放位置	m
Y_{qb}	烟幕起爆位置	m
$D(t)$	t时刻导弹位置	m
$Y(t)$	t时刻烟幕云团中心位置	m
$W(t)$	t时刻导弹位置	m
$R_z(t)$	t时刻遮蔽率	/
T_z	有效遮蔽总时长	s
θ	无人机飞行方向角（与x轴夹角）	rad
$d_{\text{弹-目}}(t)$	t时刻导弹与真目标的距离	m

五、问题一模型的建立与求解

5.1. 数据预处理

5.1.1. 坐标系建立与物理参数设定

根据题意，建立以x轴为正向，逆时针方向为正的三维直角坐标系，其中竖直向上为z轴正方向。在坐标系中，假目标位置为原点 $O(0,0,0)$ ，真目标为一个半径 $r_{mb}=7\text{ m}$ 、高度 $h_{mb}=10\text{ m}$ 的圆柱形固定目标，其底面圆心位于 $(0,200,0)$ 。

关键物理参数包括导弹飞行速度 $v_d=300\text{ m/s}$ 、烟幕云团下沉速度 $v_y=3\text{ m/s}$ 、有效遮蔽半径 $r_{yx}=10\text{ m}$ 、烟幕浓度有效遮蔽时间 $t_{yx}=20\text{ s}$ 以及重力加速度[3] $g=9.8\text{ m/s}^2$ 。

5.1.2. 初始状态参数确定

在警戒雷达发现目标时刻，各关键实体的空间位置已经确定。导弹 M1 的初始位置为 $D_0=(20000, 0, 2000)$ ，该位置表明导弹距离假目标20公里，处于相当高的威胁级别。

无人机 FY1 的初始位置为 $W_0=(17800, 0, 1800)$ 。给定的参数包括无人机飞行速度 $v_w=120\text{ m/s}$ 、投放延迟时间 $t_{tf}=1.5\text{ s}$ 以及起爆延迟时间 $t_{qb}=3.6\text{ s}$ 。则关键时间节点：投放时刻 $t_1=t_{tf}=1.5\text{ s}$ ，起爆时刻 $t_2=t_1+t_{qb}=5.1\text{ s}$ 。

5.2. 固定参数烟幕干扰模型

5.2.1. 决策变量的确定

由于问题一是给定参数的分析问题，则决策变量就是由题目条件完全确定的固定值。无人机的飞行方向根据题目要求朝向假目标方向，因此飞行方向向量可以通过起始位置和目标位置的向量差来计算。

设无人机的水平飞行方向向量为 $\vec{f}_w$ ，则根据向量计算原理有：

$$\vec{f}_w = \frac{O - W_0}{|O - W_0|} = \frac{(0,0,0) - (17800,0,1800)}{|(0,0,0) - (17800,0,1800)|} = \frac{(-17800,0,0)}{\sqrt{17800^2}} = (-1,0,0) \quad (\text{公式1})$$

由于无人机采用等高度飞行模式，因此方向向量的z分量为0，这表明无人机沿x轴负方向进行水平直线飞行。

5.2.2. 目标函数构建与推导

问题一的目标是计算在给定参数条件下烟幕干扰弹对导弹 M1 的有效遮蔽时长。目标函数可以表示为遮蔽时间的积分形式：

$$T_z = \int_{t_{start}}^{t_{end}} I_z(t) dt ,$$

其中 t_{start} 为有效遮蔽的起始时刻, t_{end} 有效遮蔽的终止时刻, I_z 为时刻 t 的遮蔽指示函数, 当存在有效遮蔽时取值为 1, 否则为 0。有效遮蔽的时间窗口为 $[t_2, t_2 + t_{yx}]$, 即从起爆时刻开始持续 20 秒。在数值计算中, 积分转化为离散求和形式:

$$T_z = \sum_{t=t_2}^{t_2+t_{yx}} \Delta t \cdot I(R_z(t)=1.0),$$

其中 $\Delta t = 0.01s$ 为时间步长, $R_z(t)$ 为时刻 t 的遮蔽率, $I(\cdot)$ 为指示函数。

5.2.3. 约束条件设定

问题一的约束条件主要体现在物理合理性和几何一致性两个方面。运动学约束要求无人机必须严格按照等高度水平飞行模式运动, 重力作用约束要求烟幕干扰弹在投放后在竖直方向做自由落体运动, 其竖直位移遵循 $\Delta z = -\frac{1}{2}gt^2$ 的规律, 这要求起爆位置的 z 坐标必须大于 0 以确保物理合理性。时间逻辑约束要求起爆时刻与投放时刻满足 $t_2 = t_1 + t_{qb}$ 的关系。遮蔽有效性约束限定烟幕有效时间为 $t \in [t_2, t_2 + t_{yx}]$ 且烟幕未落地条件为 z 时刻烟幕云团中心 z 坐标 $Y(t).z \geq 0$ 。

5.3. 运动轨迹精确计算模型

5.3.1. 无人机飞行轨迹建模

无人机 FY1 采用等高度匀速直线飞行模式, 其运动轨迹可以用参数方程精确描述。在时刻 t , 无人机的位置向量为:

$$\vec{P}_w(t) = W_0 + v_w \cdot t \cdot \vec{f}_w,$$

其中 W_0 为无人机初始位置, v_w 为飞行速度, $\vec{f}_w$ 为无人机方向向量, 具体见 (公式1)。

代入具体数值, 无人机在投放时刻 $t_1 = 1.5s$ 的位置为:

$$W(1.5) = W_0 + v_w \cdot t_1 \cdot \vec{f}_w = (17620, 0, 1800)$$

结果表明无人机在 1.5 秒内沿 x 轴负方向飞行了 180 米到达投放位置。

5.3.2. 烟幕干扰弹弹道轨迹分析

烟幕干扰弹的运动可以分为两个阶段进行分析。第一阶段为随载飞行阶段, 即从系统启动到投放时刻 t_1 , 干扰弹与无人机保持相同的运动状态, 位置向量为:

$$\vec{P}_d(t) = \vec{P}_w(t), \quad t \in [0, t_f]$$

第二阶段为平抛运动阶段, 从投放时刻 t_1 到起爆时刻 t_2 , 干扰弹在水平方向保持惯性运动, 同时在竖直方向受重力作用 (假设1)。

设投放时刻干扰弹的位置为 G_1 ，则平抛运动阶段干扰弹的水平位移为：

$$\Delta x = v_w \cdot t_{qb} \cdot \vec{f}_w, \text{ 垂直下落距离为: } \Delta z = \frac{1}{2} g t_{qb}^2。$$

起爆位置为： $Y_{qb} = G_1 + (\Delta x_x, \Delta x_y, -\Delta z)$ ，代入具体数值得 $Y_{qb} = (17188, 0, 1736.1)$ 。

5.3.3. 烟幕云团演化动态模型

烟幕干扰弹起爆后瞬时形成球状烟幕云团，随后以恒定速度 $v_{yt} = 3\text{m/s}$ 匀速下沉。云团的运动轨迹遵循简单的线性下沉规律，在时刻 $t \geq t_2$ 时，云团中心位置为：

$$Y(t) = (Y_{qb} \cdot x, Y_{qb} \cdot y, Y_{qb} \cdot z - v_{yt} \cdot (t - t_2)), \quad (\text{公式2})$$

数据代入（公式2）得： $Y(t) = (17188, 0, 1736.1 - 3 \cdot (t - 5.1))$ 。

5.3.4. 导弹飞行轨迹精确建模

导弹 M1 采用直指假目标的匀速直线飞行模式，其运动轨迹具有高度的确定性和可预测性。时刻 t ，导弹的位置 $D(t) = D_0 + v_d \cdot t \cdot \vec{f}_d$ ，导弹的飞行方向向量为：

$$\vec{f}_d = \frac{O - D_0}{|O - D_0|} = \frac{(0, 0, 0) - (20000, 0, 2000)}{|(0, 0, 0) - (20000, 0, 2000)|} \approx (-0.995, 0, -0.0995)$$

5.4. 遮蔽效果几何判断模型

5.4.1. 圆柱体目标离散化建模

真目标作为圆柱体需要进行适当的离散化处理以便于进行遮蔽效果的数值计算。采用分层采样策略，在圆柱体的关键几何位置选取采样点。

底面圆周采样点为：

$$P_{d,i} = \left(r_{mb} \cos\left(\frac{2\pi i}{8}\right), 200 + r_{mb} \cos\left(\frac{2\pi i}{8}\right), 0 \right),$$

顶面圆周采样点为：

$$P_{t,i} = \left(r_{mb} \cos\left(\frac{2\pi i}{8}\right), 200 + r_{mb} \cos\left(\frac{2\pi i}{8}\right), h_{mb} \right),$$

轴线采样点为：

$$P_{z,i} = \left(0, 200, \frac{j \cdot h_{mb}}{4} \right)$$

其中 r_{mb} 为真目标半径、 h_{mb} 为真目标高度。这种采样策略确保了对圆柱体表面的全面覆盖。

5.4.2. 线段与球面相交算法推导

遮蔽效果核心判断依据是将导弹视为一点（假设2），判断导弹到目标采样点视线是否被烟幕云团阻挡（假设3）。这个问题可以转化为判断三维空间中线段与球面是否相交的几何问题。设导弹位置为 $D(t)$ ，目标采样点为 $\vec{P}$ ，烟幕云团中心位置为 $Y(t)$ ，有效半径为 r_{yx} ， s 表示从线段起点（导弹位置）到导弹到目标点之间的线段上某点的相对距离比例， s_0 是用于定位线段上距离烟幕云团中心最近点的参数值，连接导弹和目标点的线段可以用参数方程表示为：

$$\vec{L}(s) = D(t) + s(\vec{P} - D(t)), \quad s \in [0, 1]$$

设线段方向向量为 $\vec{x} = \vec{P} - D(t)$ ，单位向量为 $\hat{x} = \frac{\vec{x}}{|\vec{x}|}$ ，球心到线段起点的向量为 $\vec{y} = Y(t) - D(t)$ 。线段上距离球心最近的点对应的投影长度为：

$$h = \hat{y} \cdot \hat{x} = \frac{\vec{y} \cdot \vec{x}}{|\vec{x}|^2},$$

其中 $\vec{x} = \vec{P} - \vec{D}$ ， $\vec{y} = \vec{Y}_{yt} - \vec{D}$ ， $\vec{P}$ 为真目标表面采样点的位置向量， $\vec{D}$ 为导弹在时刻 t 的位置向量， $\vec{Y}_{yt}$ 为烟幕云团中心在时刻 t 的位置向量，线段与球面相交的充要条件是最短距离 $d \leq r_{yx}$ 。

当 $h \in [0, |\vec{x}|]$ 时，最近点在线段上，此时最近点为： $\vec{L}_{最近} = D(t) + \frac{h}{|\vec{x}|} \cdot \vec{x}$ 最短距离为： $d = |Y(t) - \vec{L}_{最近}|$ 。

当 $s_0 \in [0, 1]$ 时，最近点在线段内部，最短距离为： $d = |\vec{Y}_{yt} - (\vec{D} + s_0 \vec{x})|$ 。

当 $h \notin [0, |\vec{x}|]$ 时，最近点为线段端点，最短距离为： $d = \min(|Y(t) - D(t)|, |Y(t) - \vec{P}|)$ 。

当 $h \in [0, |\vec{x}|]$ 时，最近点为线段端点，最短距离为： $d = \min(|\vec{Y}_{yt} - \vec{D}|, |\vec{Y}_{yt} - \vec{P}|)$ 。

5.4.3. 遮蔽效果综合评估模型

对于给定时刻 t ，需要遍历所有目标采样点，统计被有效遮蔽的点数比例。设总采样点数为 N ，被遮蔽的点数为 N_z ，则遮蔽率为：

$$R_z(t) = \frac{N_z}{N} \quad (\text{公式3})$$

考虑到实际应用中对遮蔽效果的严格要求，采用100%遮蔽率作为有效遮蔽的判断标准，即只有当 $R_z(t) = 1.0$ 时才认为在时刻 t 存在有效遮蔽。这种严格的判断标准确保了遮蔽效果的可靠性，避免了部分遮蔽可能带来的不确定性。

5.5. 模型求解与结果分析

5.5.1. 数值计算方法设计

考虑到遮蔽效果判断的复杂性和对计算精度的要求，采用时间离散化方法进行数值求解。在有效时间窗口 $[t_2, t_2+t_{yx}] = [5.1, 25.1]$ 内，以 $\Delta t = 0.01s$ 为时间步长进行遍历计算。对于每个时间节点，根据前述的运动轨迹模型计算导弹位置 $D(t)$ 和烟幕云团位置 $Y(t)$ ，然后通过线段与球面相交算法判断所有采样点的遮蔽情况。计算过程中特别关注遮蔽状态发生转换的时刻，记录从无遮蔽到有遮蔽以及从有遮蔽到无遮蔽的精确时间点，并累计所有有效遮蔽时间段的长度。有效遮蔽时间通过时间离散化方法计算：

$$T_z = \sum_{t_i=t_{qb}}^{t_{qb}+t_{yx}} \Delta t \cdot I(R_z(t_i) = 1.0),$$

其中 t_{yx} 为烟幕有效持续时间， t_i 为 $[t_{qb}, t_{qb} + t_{yx}]$ 中第 i 个时间节点。 $I(\cdot)$ 为指示函数，当条件满足时取值为 1，否则为 0， $R_z(t_i)$ 见（公式3）。

5.5.2. 关键结果数据分析

通过数值计算得到问题一的完整解。投放时刻为 1.5 秒，此时无人机位于 (17620, 0, 1800)，投放的烟幕干扰弹经过 3.6 秒的平抛运动后在 $t_2=5.1s$ 秒时刻于位置 (17188, 0, 1736.1) 起爆。起爆后形成的烟幕云团在有效持续时间 20 秒内，对导弹 M1 产生了总计约 1.392 秒的有效遮蔽时间。

六、问题二模型的建立与求解

6.1. 数据预处理

6.1.1. 优化参数空间定义

问题二在问题一的基础上，将原本固定的参数转化为优化变量，形成一个四维优化问题。优化参数包括无人机飞行速度 $v_w \in [70, 140]m/s$ 、投放延迟时间 $t_{yf} \in [0.1, 4.0]s$ 、起爆延迟时间 $t_{qb} \in [0.5, 4.0]s$ 以及飞行方向角 $\theta \in [0, 2\pi]rad$ 。飞行速度的约束反映了无人机的性能限制，时间参数的约束确保了系统响应的及时性，而飞行方向的全角度范围则为找到最优拦截轨迹提供了最大的灵活性。

设优化变量向量为：

$$\vec{x} = (v_w, t_{yf}, t_{qb}, \theta) \quad (\text{公式4})$$

6.1.2. 优化目标重新定义

与问题一的计算目标不同，问题二的核心目标是通过参数优化使有效遮蔽时间最大化[4]。优化目标函数可以表述为：

$$\max f(\vec{x}) = T_z(\vec{x}),$$

其中， $T_z(\vec{x})$ 为在给定参数配置下的有效遮蔽时间， $\vec{x}$ 见（公式4）。

6.2. 多参数优化烟幕干扰模型

6.2.1. 决策变量的确定

问题二的决策变量构成了一个四维连续优化空间，每个变量都对最终的遮蔽效果产生重要影响。优化向量 $\vec{x} = (v_w, t_{yf}, t_{qb}, \theta)$ 中，无人机的飞行方向向量表示为：

$$\vec{f}_w = (\cos \theta, \sin \theta, 0)$$

6.2.2. 目标函数构建与推导

问题二的目标函数需要综合考虑遮蔽效果最大化和实际约束条件。基础目标函数为遮蔽时间最大化，但为了确保优化结果的实际可行性，需要将其转化为最小化问题并引入惩罚函数 $P_{cf}(\vec{x})$ ：

$$\max f(\vec{x}) = -T_z(\vec{x}) + P_{cf}(\vec{x}),$$

其中 $T_z(\vec{x})$ 的计算同问题一，但位置参数随 $\vec{x}$ 变化。投放位置计算为： $W_1 = W_0 + v_w \cdot t_{yf} \cdot \vec{f}_w$

起爆位置计算为： $Y_{qb} = W_1 + \left(v_w \cdot t_{qb} \cdot \cos \theta, v_w \cdot t_{qb} \cdot \sin \theta, -\frac{1}{2} g t_{qb}^2 \right)$ 。

惩罚函数 $P_{cf}(\vec{x})$ 包含多个惩罚项：

$$P_{cf}x = \begin{cases} (0.5t_{yf} + t_{qb} - 10) & \text{若 } t_{yf} + t_{qb} > 10 \\ \frac{d_{fx} - 1500}{1000} & \text{若 } d_{fx} > 1500 \\ 1000 & \text{若违反其他约束} \end{cases},$$

其中 $d_{fx} = |W_1 - W_0| = v_w \cdot t_{yf}$ 为表示无人机从初始位置到投放烟幕干扰弹时的飞行距离。

6.2.3. 约束条件设定

问题二需要同时满足物理可行性、几何合理性和操作实用性的要求。参数范围约束要求 $70 \leq v_w \leq 140$ 、 $0.1 \leq t_{yf} \leq 4.0$ 、 $0.5 \leq t_{qb} \leq 4.0$ 、 $0 \leq \theta \leq 2\pi$ 。物理约束包括起爆位置高度约束 $Y_{qb,z} \geq 0$ 、总响应时间约束 $t_{yf} + t_{qb} \leq 10s$ 以及飞行距离约束 $d_{fx} \leq 1500m$ 。

具体约束条件包括：时间约束要求总响应时间不超过预设上限：

$$g_1(\vec{x}) = t_{yf} + t_{qb} - T_{\text{最大}} \leq 0,$$

飞行距离约束要求投放时刻的飞行距离不超过上限：

$$g_2(\vec{x}) = v_w \cdot t_{gf} - D_{\text{最大}} \leq 0,$$

其中 $D_{\text{最大}} = 1500\text{m}$ 为最大飞行距离限制。起爆高度约束要求起爆位置在地面以上：

$$g_3(\vec{x}) = W \cdot z(\vec{x}) \geq 0,$$

其中 $W \cdot z(\vec{x})$ 为起爆位置的 z 坐标。

6.3. 自适应优化算法模型

6.3.1. 差分进化算法设计

考虑到问题二优化空间的复杂性和目标函数的非线性特征，采用差分进化算法[5]作为主要的全局优化方法。算法的核心进化操作包括变异、交叉和选择三个步骤。变异操作通过以下公式生成变异向量：

$$\vec{v}_{i,g+1} = \vec{x}_{r1,g} + F(\vec{x}_{r2,g} - \vec{x}_{r3,g}),$$

其中 $F=0.8$ 为变异因子， $r1, r2, r3$ 为随机选择的不同个体索引。交叉操作按照交叉概率 $CR = 0.9$ 在当前个体 $\vec{x}_{i,g}$ 和变异向量 $\vec{v}_{i,g+1}$ 之间进行基因交换：

$$u_{j,i,g+1} = \begin{cases} v_{j,i,g+1} & \text{若 } rand_{j,i} \leq CR \text{ 或 } j = j_{rand} \\ x_{j,i,g+1} & \text{否则} \end{cases},$$

选择操作基于目标函数值进行贪婪选择，保留适应度更高的个体进入下一代。算法设置种群规模为 30，最大迭代次数为 100，收敛精度为 10^{-6} 。

6.3.2. 自适应时间步长策略

遮蔽效果计算的精度直接影响优化结果的可靠性，因此需要采用自适应时间步长策略来平衡计算精度和效率。基本策略是根据导弹与目标的距离动态调整时间步长：

$$\Delta t = \begin{cases} \Delta t_{\text{精细}} = 0.005 \text{ s} & \text{若 } |(t) - T_{\text{中心}}| < 100\text{m} \\ \Delta t_{\text{基础}} = 0.02 \text{ s} & \text{否则} \end{cases},$$

其中 $T_{\text{中心}} = (0, 200, 5)$ 为真目标中心位置。

这种自适应策略确保在导弹接近目标的关键阶段采用更精细的时间分辨率，在远离阶段采用较大步长提高计算效率。

6.3.3. 收敛性判断与结果验证

优化算法的收敛性通过多个指标进行综合判断。目标函数值收敛条件为连续多代的最优值变化小于设定阈值 10^{-6} 。种群多样性收敛通过计算种群个体间的距离方差来评估：

$$\sigma^2 = \frac{1}{Z} \sum_{i=1}^N |\vec{x}_i - \bar{\vec{x}}|^2,$$

其中 $Z=30$ 为种群规模， $\bar{x}$ 为种群中心。约束满足度检验确保最优解满足所有物理和几何约束。为了验证结果的可靠性，采用多次独立运行的策略，比较不同随机种子下的优化结果。

6.4. 高精度遮蔽计算模型

6.4.1. 增强型几何相交算法

问题二在问题一算法基础上，进一步优化了线段与球面相交判断的数值稳定性。特别是在处理边界情况时，采用了更严格的数值误差控制，避免了浮点运算误差对结果的影响。算法还引入了早期终止机制，当检测到某个时刻的遮蔽率明显偏低时，可以提前判断该参数组合的劣势，避免完整计算所有时间节点。

6.4.2. 遮蔽质量评估体系

建立遮蔽质量评估体系来更全面地评价不同参数配置的优劣。遮蔽质量评估包括遮蔽连续性、遮蔽强度和遮蔽时机三个维度，各维度量化指标如下：

遮蔽连续性指标：定义遮蔽片段数量 K 和平均片段长度 $\bar{L}$ ：

$$\bar{L} = \frac{T_z}{K}$$

其中 T_z 为总遮蔽时间。连续性得分 S_1 为： $S_1 = \exp\left(-\frac{K}{T_z}\right)$ ，当 K 越小、 $\bar{L}$ 越大， S_1 越接近 1，连续性越好。

遮蔽强度指标：计算全时段平均遮蔽率 $\bar{R}$ ：

$$\bar{R} = \frac{1}{t_z} \int_{t_{start}}^{t_{end}} R_z(t) dt,$$

其中 t_z 为烟幕有效时间为 20s， t_{start} 、 t_{end} 分别为有效时间窗口的起止时刻。强度得分 S_2 为： $S_2 = \bar{R}^2$ ， $\bar{R}$ 越接近 1， S_2 越接近 1，强度越高。

遮蔽时机指标：定义导弹与真目标距离 $d_{\text{弹目}}(t) = |W(t) - T_{\text{中心}}|$ ，关键时机权重函数：

$$w(t) = \exp\left(-\frac{d_{\text{弹目}}(t)}{1000}\right),$$

时机得分 S_3 为加权遮蔽率积分： $S_3 = \frac{\int_{t_{start}}^{t_{end}} R_z(t) \cdot w(t) dt}{\int_{t_{start}}^{t_{end}} w(t) dt}$ ，综合质量得分 S 为三指标加权

和： $S = 0.3S_1 + 0.3S_2 + 0.4S_3$ 。

6.5. 模型求解与结果分析

通过求解最终得到最优遮蔽时长为 4.82s，对应飞行角度 52.95°、无人机速度 119.31m/s、投放延迟 0.1110s、起爆延迟 1.5209s。

七、问题三模型的建立与求解

7.1. 数据预处理

7.1.1. 坐标系建立与物理参数设定

问题三要求在问题一建立的坐标系基础上，扩展为多枚烟幕干扰弹的协同干扰模型。坐标系仍以假目标为原点，水平面为 xy 平面，竖直向上为 z 轴正方向，关键物理参数保持与问题一相同。

7.1.2. 多弹投放约束参数确定

问题三涉及3枚烟幕干扰弹的投放，需要考虑投放间隔约束。设定最小投放间隔 $\Delta t_{\min} = 1.0\text{s}$ ，确保相邻两枚烟幕弹的投放时间间隔满足：

$$t_{tf,i+1} - t_{tf,i} \geq \Delta t_{\min},$$

其中 $t_{tf,i}$ 为第 i 枚烟幕弹的投放时间， $t_{tf,i+1}$ 为第 $i+1$ 枚烟幕弹的投放时间。

7.2. 多枚烟幕弹协同干扰模型

7.2.1. 决策变量的确定

问题三的决策变量构成一个 8 维优化空间，每枚烟幕弹有 4 个独立的决策变量。设优化变量向量为：

$$\vec{x} = [v_w, t_{tf,1}, t_{qb,1}, \theta, t_{tf,2}, t_{qb,2}, t_{tf,3}, t_{qb,3}], \quad (\text{公式5})$$

其中对于第 i 枚烟幕弹 ($i = 1, 2, 3$)： θ 为飞行方向角，第 i 枚烟幕弹的无人机飞行方向向量为： $\vec{f}_{w,i} = [\cos(\theta_i), \sin(\theta_i), 0]$ ， v_w 为无人机飞行速度， $t_{tf,i}$ 为投放延迟时间， $t_{qb,i}$ 为起爆延迟时间。

7.2.2. 目标函数构建与推导

问题三的目标是最大化 3 枚烟幕弹对导弹 M1 的总有效遮蔽时间。由于多枚烟幕弹的遮蔽时间可能存在重叠，需要考虑时间段的合并处理。

设第 i 枚烟幕弹的遮蔽时间段集合为：

$$\vec{f}_{y,i} = \{[t_{start,i,j}, t_{end,i,j}]\},$$

其中 j 表示该烟幕弹产生的第 j 个连续遮蔽时间段。所有遮蔽时间段的集合为： $T_z = T_1 \cup T_2 \cup T_3$ 。对重叠的时间段进行合并处理，得到不重叠的时间段集合 $T_{merged} = \{[t_{start,k}, t_{end,k}]\}$ 。目标函数为：

$$\max f(\vec{x}) = \sum_k (t_{end,k} - t_{start,k}),$$

其中合并算法采用区间排序和贪心合并策略：首先将所有时间段按起始时间排序，其次逐个合并重叠或相邻的时间段，最后计算合并后总时间长度， $\vec{x}$ 见（公式5）。

7.2.3. 约束条件设定

问题三的约束条件包括参数范围约束、时间逻辑约束和物理可行性约束。参数范围约束要求 $70 \leq v_i \leq 140$, $i=1,2,3$ 、 $0.1 \leq t_{f,i} \leq 10.0$, $i=1,2,3$ 、 $1.0 \leq t_{qb,i} \leq 10.0$, $i=1,2,3$ 、 $0 \leq \theta_i \leq 2\pi$, $i=1,2,3$ 。时间逻辑约束包括 $t_{f,i+1} - t_{f,i} \geq \Delta t_{\min} = 1.0$, $i=1,2$ ，物理可行性约束包括起爆位置高度约束 $Y_{qb,Z,i} = Y_{f,Z,i} - \frac{1}{2}gt_{qb,i}^2 \geq 0$ ，其中 $Y_{f,Z,i}$ 表示投放位置的 z 坐标。

7.3. 分层扫描计算模型

7.3.1. 粗-细两阶段扫描策略

为提高遮蔽时间计算效率，采用分层扫描策略。第一阶段是粗扫描定位，时间步长设为0.2秒，目的是快速定位潜在遮蔽区域，覆盖范围为从0到最大时间的全时间域。在粗扫描阶段，每个时间点按0.2秒的间隔依次选取，检查是否存在任意一枚烟幕弹能够产生有效遮蔽，若检测到遮蔽，就将该时间点标记为潜在区域。第二阶段是细扫描精确化，时间步长设为0.05秒，目的是在潜在区域内精确计算遮蔽时间，覆盖范围为粗扫描发现的候选区域前后各0.2秒。对于每个潜在区域，在该区域前后0.2秒的区间内进行细扫描，以精确确定遮蔽的起止时间。

7.3.2. 轨迹计算与遮蔽判断

对于第 i 枚烟幕弹，其轨迹计算与问题一类似，但需要考虑不同的飞行参数，具体参数为：

投放位置： $\vec{Y}_{f,i} = \vec{W}_0 + v_i \cdot t_{f,i} \cdot \vec{f}_{w,i}$ ，其中 $\vec{Y}_{f,i}$ 为 FY1 的初始位置。

起爆位置： $\vec{Y}_{qb,i} = \vec{Y}_{f,i} \vec{W}_0 + \left[v_i \cdot t_{f,i} \cdot \cos(\theta), v_i \cdot t_{qb,i} \cdot \sin(\theta), -\frac{1}{2}gt_{qb,i}^2 \right]$ 。

烟幕云团动态位置：在时刻 $t \geq t_{bomb,i} = t_{f,i} + t_{qb,i}$ 时，第 i 个云团的位置为：

$$\vec{Y}_i(t) = \vec{Y}_{qb,i} + [0, 0, -v_{yt}(t - t_{bomb,i})],$$

其中 $t_{bomb,i}$ 表示第 i 枚烟幕弹的起爆时刻， v_{yt} 表示烟幕云团的下沉速度。

7.3.3. 综合遮蔽效果判断

在任意时刻 t ，需要判断所有有效的烟幕云团对目标的综合遮蔽效果。设在时刻 t ，有效的云团集合为： $I_{eff}(t) = \{i | t_{bomb,i} \leq t \leq t_{bomb,i} + t_{yx} \text{ 且 } \vec{Y}_{i,z}(t) > 0\}$ 。

对于真目标的每个采样点 $\vec{N}_j$ ，其被遮蔽的条件为：存在至少一个有效云团使得导弹到目标点的视线被阻挡：

$$blocked_j(t) = \forall_{i \in I_{eff}(t)} LineIntersectSphere(\vec{D}(t), \vec{N}_j, \vec{Y}_i(t), r_{yx})$$

时刻 t 的总遮蔽率为: $R_z(t) = \frac{1}{N_{sample}} \sum_{j=1}^{N_{sample}} blocked_j(t)$ 当 $R_z(t) = 1.0$, 时, 认为在时

刻 t 存在有效遮蔽。

7.4. 增强遗传算法优化模型

7.4.1. 算法参数设定

考虑到8 维优化空间的复杂性, 采用增强遗传算法[2]进行全局优化。算法参数设置如下: 种群规模为20, 最大迭代次数100次, 变异因子 $F=0.15$, 交叉概率 $CR = 0.9$, 早停设置为50轮结果无优化即停。

7.4.2. 适应度函数设计

将最大化问题转换为最小化问题, 并引入惩罚函数处理约束:

$$\min g(\vec{x}) = -f(\vec{x}) + P(\vec{x}),$$

其中惩罚函数 $P(\vec{x})$ 包含多项惩罚: $P(\vec{x}) = \alpha_1 P_t(\vec{x}) + \alpha_2 P_d(\vec{x}) + \alpha_3 P_z(\vec{x})$ 。

时间约束惩罚: $P_t(\vec{x}) = \max(0, \max_i t_{ff,i} + t_{qb,i} - 15)$ 。

距离约束惩罚: $P_d(\vec{x}) = \sum_{i=1}^3 \max(0, \|\vec{Y}_{ff,i} - \vec{W}_0\|_2 - 2000)$ 。

高度约束惩罚: $P_z(\vec{x}) = \sum_{i=1}^3 \max(0, -Y_{ff,i,z}) \times 100$ 。

7.4.3. 启发式个体生成策略

采用基于问题特性的智能个体生成方法, 无人机参数飞行方向在 $[0, 360^\circ)$ 内均匀随机取值, 飞行速度 $[70, 140]$ m/s 内均匀随机取值。

第 i 枚烟幕弹的生成策略为投放时间:

$$t_{ff,i} = t_{base} + i \cdot \Delta t_{interval},$$

其中 t_{base} 在 $[0.5, 2.0]$ s 内取值, $\Delta t_{interval}$ 在 $[1.2, 3.0]$ s 内取值。

根据投放顺序差异化设置第 1 枚烟雾弹投放时间在 $[2.0, 4.0]$ s 内取值, 第 2 枚烟雾弹投放时间在 $[3.0, 6.0]$ s 内取值, 第 3 枚投放时间在 $[4.0, 8.0]$ s 内取值。

7.4.4. 自适应进化策略

设置变异强度随进化进程动态调整, 以使得早期变异较强以保证充分探索可能的解, 后期变异较弱以确保算法能够收敛到较好的解:

$$\sigma_m(g) = 1.0 - 0.8 \cdot \frac{g}{G_{max}},$$

其中 g 为当前代数, G_{max} 为最大代数。

采用算术交叉的方式避免过度复杂化，通过一个在0.3到0.7之间取值的参数，对两个父代个体进行组合生成两个子代个体，从而避免产生极端值。

进化算法的选择机制中，“锦标赛选择”[6]是一种常用的策略。具体来说，就是从种群中随机挑选一定数量的个体组成一个“锦标赛小组”，然后在这个小组里选择适应度最高的个体进入下一代种群[7]。本文在使用锦标赛选择的同时引入概率机制，避免过度偏向当前最优解。对于锦标赛适应度向量 $f_{tournament}$ 选择概率 $p_i = \frac{f_i}{\sum_j f_j}$ ，从而保持种群多样性，避免早熟收敛。

7.4.5. 算法进化过程分析

在增强遗传算法迭代优化过程中，最佳适应度与平均适应度的变化趋势如图 1 所示。由图可知，随着迭代代数的增加，最佳适应度逐步提升并趋于稳定，在约 80 代时达到 6 左右，表明算法能够有效搜索到较优解；平均适应度同步提高，反映种群整体质量不断改善。

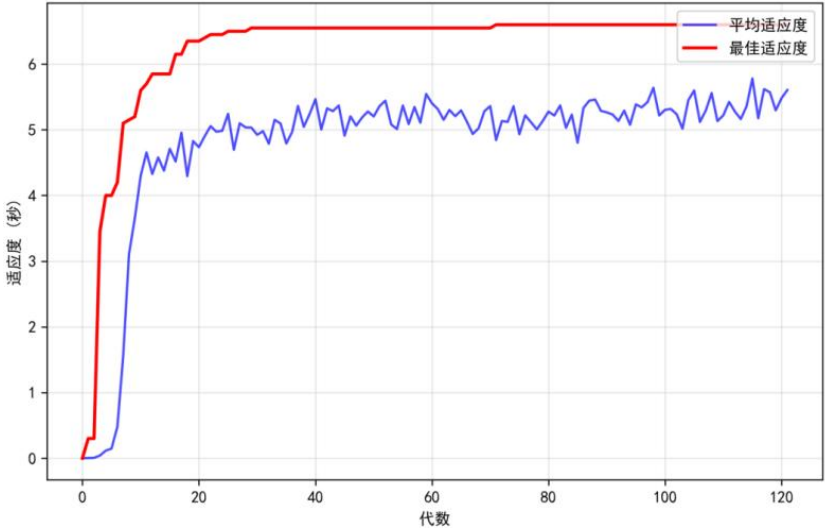


图 1 适应度演化过程

7.5. 模型求解与结果分析

7.5.1. 优化结果

通过增强遗传算法的迭代求解，最终获得了一组最优参数配置如表1：

表 1 问题三结果

烟幕弹编号	飞行方向(度)	飞行速度(m/s)	投放时间(s)
1	179.5	112.5	0.51
2	179.5	112.5	3.77
3	179.5	112.5	5.3

7.5.2. 烟幕弹时间协同分析

三枚烟幕弹的时间配置呈现出有序衔接的特点（如图2），第一枚烟幕弹在4.1s进入有效期，第二枚在8.5-28.5s区间发挥作用，第三枚则覆盖10.6-30.6s时段，通过错峰投放实现了遮蔽时间的连续覆盖，减少了时间空白区。

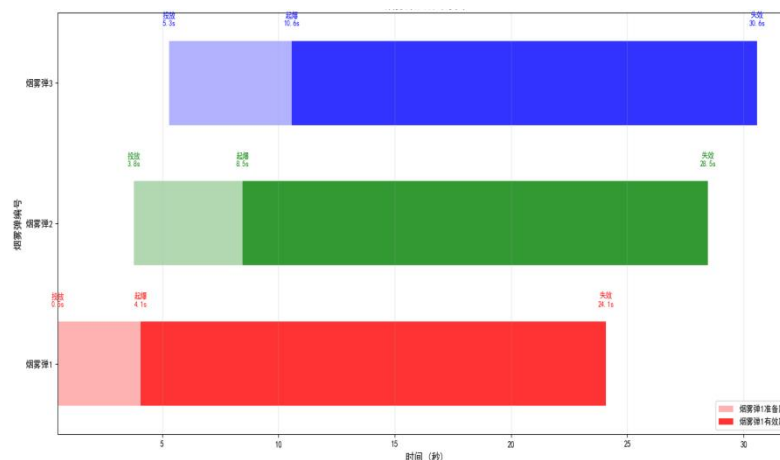


图2 烟幕弹有效期时序图

7.5.3. 空间轨迹协同分析

从空间轨迹来看（如图3），三枚烟幕弹的投放点与起爆点沿导弹飞行路径依次分布，形成纵深拦截态势。烟幕弹1起爆点靠近无人机初始位置 $x \approx 17500\text{m}$ ，烟幕弹2起爆点在 $x \approx 11000\text{m}$ 处，烟幕弹3起爆点在 $x \approx 5000\text{m}$ 处，与导弹逐步接近假目标的轨迹形成动态匹配，确保在不同阶段均能形成有效遮蔽。

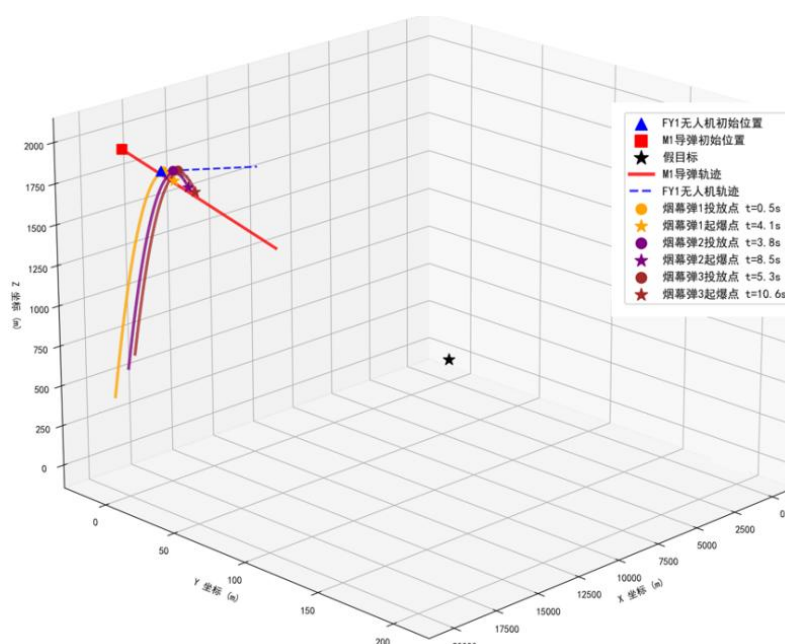


图3 无人机与烟幕弹空间轨迹

八、问题四模型的建立与求解

8.1. 数据预处理

8.1.1. 多无人机协同系统建模

问题四为三架不同无人机的协同干扰模型，采用多机单弹的分布式协同策略。坐标系建立与物理参数设定与问题三保持一致，关键区别在于需要考虑不同无人机的初始位置差异对协同效果的影响。

8.1.2. 无人机初始配置参数

三架无人机 FY1、FY2、FY3 需要协同作战，其初始位置分别为：FY1:(17800, 0, 1800)，FY2:(12000, 1400, 1400)，FY3: (6000, -3000, 700)，三架无人机在空间上呈现分层分布的态势。

8.2. 多机协同烟幕干扰模型

8.2.1. 决策变量的确定

问题四的决策变量构成一个 12 维优化空间，每架无人机有 4 个独立的决策变量。设优化变量向量为：

$$\vec{x} = [v_{w,1}, t_{f,1}, t_{qb,1}, \theta_1, v_{w,2}, t_{f,2}, t_{qb,2}, \theta_2, v_{w,3}, t_{f,3}, t_{qb,3}, \theta_3], \quad (\text{公式6})$$

其中对于第 i 架无人机 ($i = 1, 2, 3$)： θ 为飞行方向角， $v_{w,i}$ 为飞行速度， $t_{f,i}$ 为投放延迟时间， $t_{qb,i}$ 为起爆延迟时间，飞行方向向量为： $\vec{f}_{w,i} = \left[\cos\left(\frac{\theta_i \pi}{180}\right), \sin\left(\frac{\theta_i \pi}{180}\right), 0 \right]$

8.2.2. 目标函数构建与推导

问题四的目标是最大化 3 架无人机各投放 1 枚烟幕弹对导弹 M1 的总有效遮蔽时间。由于多架无人机的遮蔽时间可能存在重叠，需要采用时间段合并策略处理重叠问题。设第 i 架无人机产生的遮蔽时间段集合为：

$$T_i = \{[t_{start,i,j}, t_{end,i,j}]\}$$

其中 $t_{start,i,j}$ 和 $t_{end,i,j}$ 分别表示第 i 枚烟幕弹产生的第 j 个连续遮蔽时间段的起始时刻和结束时刻。

所有遮蔽时间段的集合为： $T_z = T_1 \cup T_2 \cup T_3$ ，目标函数为：

$$\max f(\vec{x}) = \sum_{k=1}^K (t_{end,k} - t_{start,k}), \quad (\text{公式7})$$

其中 K 为合并后不重叠时间段的数量， $t_{start,k}$ 和 $t_{end,k}$ 分别表示经过合并处理后第 K 个不重叠遮蔽时间段的起始时刻和结束时刻。

8.2.3. 约束条件设定

约束条件包括参数范围约束、物理可行性约束。参数范围约束要求：

$$\begin{cases} 70 \leq v_{w,i} \leq 140, & i = 1, 2, 3 \\ 1.0 \leq t_{qb,i} \leq 15.0, & i = 1, 2, 3 \\ 0 \leq \theta_i \leq 2\pi, & i = 1, 2, 3 \\ t_{tf,i} \geq 0, & i = 1, 2, 3 \\ Y_{qb,Z,i} \geq 0 \end{cases},$$

其中对于第*i*架无人机： $Y_{tf,Z,i}$ 表示投放的烟幕弹起爆位置*z*坐标， $t_{tf,i}$ 表示投放延迟时间， $t_{qb,i}$ 表示起爆延迟时间， $v_{w,i}$ 表示无人机速度， θ_i 表示飞行角度。

8.3. 增强遗传算法优化模型

8.3.1. 算法框架设计

考虑到12维优化空间的复杂性和多机协同的耦合效应，采用增强遗传算法进行全局优化。算法参数设置如下：种群规模为150，最大迭代次数200次，变异因子 $F=0.1$ ，交叉概率 $CR=0.9$ ，精英保留规模为30。

8.3.2. 智能个体生成策略

针对多机协同的特点，设计基于几何约束的策略个体生成方法：

导弹轨迹分析：预计算导弹M1的关键时间点：

$$t_{\text{intercept}} = [15\text{s}, 30\text{s}, 45\text{s}]$$

Y轴约束修正：为避免无效投放，当预测投放点*y*坐标为负时，调整飞行方向：

$$\theta_i = \begin{cases} \text{uniform}(-30^\circ, 30^\circ) & \text{若 } W_{i,y}[1] < 0 \\ \text{uniform}(-30^\circ, 30^\circ) & \text{否则} \end{cases}$$

8.3.3. 适应度函数设计

将最大化问题转换为最小化问题，并引入多层次惩罚机制 $\min g(\bar{x}) = -f(\bar{x}) + P(\bar{x})$ ，其中惩罚函数 $P(\bar{x})$ 包含多项惩罚： $P(\bar{x}) = P_t(\bar{x}) + P_c(\bar{x}) + P_z(\bar{x})$ 。

时间约束惩罚： $P_t(\bar{x}) = 500 \times \sum_{i=1}^3 \max(0, t_{tf,i} + t_{qb,i} - 30)$ 。

约束违反惩罚： $P_c(\bar{x}) = 1000 \times \sum_{i=1}^3 C_i$ ，其中 C_i 为第*i*架无人机的约束违反次数。

高度约束惩罚： $P_z(\bar{x}) = 1000 \times \sum_{i=1}^3 \max(0, -Y_{tf,i,z})$ 。

8.3.4. 自适应进化策略

设置变异强度随进化进程动态调整，以使得早期变异较强以保证充分探索可能的解，后期变异较弱以确保算法能够收敛到较好的解： $\sigma_m(g) = \sigma_{\max} \left(1 - \frac{g}{G_{\max}} \right)^2$ ，其中 g 为当前代数， $G_{\max}$ 为最大代数。

采用算术交叉的方式避免过度复杂化，通过取0.5使得每架无人机参数有 50% 概率进行交叉，确保协同策略的多样性。

8.4. 模型求解与结果分析

无人机FY1、FY2、FY3执行烟幕干扰任务的三维飞行轨迹如图四所示，轨迹覆盖 6000-18000m 水平距离、-400-1000m 横向距离及 600-1800m 高度范围，不同无人机轨迹通过差异化标识区分，直观呈现其飞行方向与路径差异。

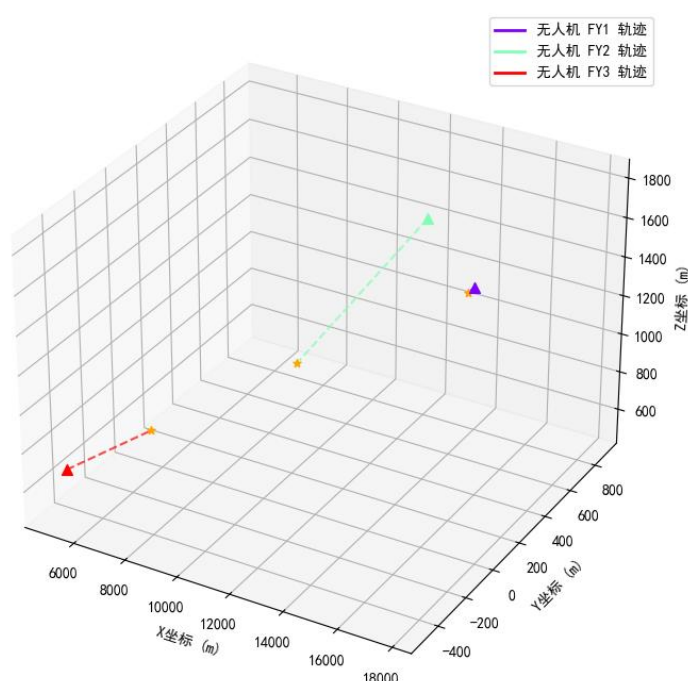


图 4 无人机与烟幕弹空间轨迹

通过增强遗传算法的迭代求解，最终获得了一组最优参数配置，在这一配置下，3 架无人机各投放1枚烟幕弹对导弹M1的总有效遮蔽时间达到了理想效果。具体的参数细节如表2：

表 2 问题四结果

无人机编号	飞行方向(度)	飞行速度(m/s)	投放时间(s)	起爆延迟(s)
FY1	178.4	84.1	0.5	2.84
FY2	238.1	140	4.39	7.14
FY3	107.8	98	26.91	6.22

九、问题五模型的建立与求解

9.1. 多导弹多无人机协同干扰模型

问题五相比前述问题的根本区别在于引入了多目标威胁和多平台协同的复合优化挑战。与单导弹场景不同，三枚导弹 M1、M2、M3 的同时来袭构成了空间 - 时间二维的威胁网络，其飞行时间分别为 66.7 秒、63.4 秒和 60.1 秒，时间差仅 6.6 秒，要求防御系统必须在极短窗口内实现全方位覆盖。

决策变量构成一个 45 维优化空间，每架无人机需要决定投放的烟幕弹数量，以及每枚烟幕弹的具体参数：

$$\vec{X} = [\vec{x}_1, \vec{x}_2, \vec{x}_3, \vec{x}_4, \vec{x}_5]^T$$

每架无人机有 4 个独立的决策变量。设优化变量向量为：

$$\vec{x}_i = [v_{w,i}, n_i, \theta_i, t_{f1,i}, t_{qb1,i}, t_{f2,i}, t_{qb2,i}, t_{f3,i}, t_{qb3,i}], \quad (\text{公式8})$$

其中，对于第 i 架无人机： θ_i 为飞行方向角，取值范围为 $[0^\circ, 360^\circ]$ 。 $v_{w,i}$ 为飞行速度，取值范围为 $[70, 140]$ 。 n_i 为第 i 架无人机实际投放的烟幕弹数量，取值为离散值 $\{1, 2, 3\}$ 。 $t_{qb1,i}, t_{qb2,i}, t_{qb3,i}$ 为第 i 架无人机第 1、2、3 枚烟幕弹的起爆延迟时间，取值范围为 $[1.0, 15.0]$ s，即从投放后到烟幕弹起爆的时间间隔。 $t_{f1,i}, t_{f2,i}, t_{f3,i}$ 为第 i 架无人机第 1、2、3 枚烟幕弹的投放时间。

9.2. 多导弹威胁态势建模

9.2.1. 累积遮蔽时间模型

多导弹[8]场景的目标函数设计超越了简单的时间累加，需要考虑多导弹的威胁程度差异和遮蔽效果的平衡性：

$$\max g(\vec{X}) = \sum_{j=1}^3 T_j(\vec{X}) + \alpha \cdot P_b(\vec{X}) + \beta \cdot P_s(\vec{X}) \quad (\text{公式9})$$

平衡性奖励项： $P_b(\vec{X}) = -Var(T_1, T_2, T_3)$ 。

协同效应项： $P_s(\vec{X}) = \sum_{j=1}^3 \sum_t \min(C_j(t), 1) \cdot \log(1 + C_j(t))$ ， $C_j(t)$ 表示时刻 t 对导弹 j 的重叠遮蔽数量。

9.2.2. 时空耦合的遮蔽判断

多导弹场景下，需要考虑烟幕弹的 "一对多" 和 "多对一" 效应，则遮蔽指示函数：

$$I_j(t, \vec{X}) = 1 \Leftrightarrow \exists i, k: Active(\vec{S}_{ik}(t)) \wedge Occlude(\vec{D}_j(t), \vec{S}_{ik}(t)),$$

其中烟幕弹有效性判断： $Active(\vec{S}_{ik}(t)) = (t \geq t_{f,i,k} + t_{qb,i,k}) \wedge (t \leq t_{f,i,k} + t_{qb,i,k}) \wedge (\vec{S}_{ik,z}(t) \geq 0)$ ， $\vec{S}_{ik}(t)$ 表示第 i 个烟幕球在时刻 t 的位置。

9.3. 复合约束体系与智能修复

9.3.1. 分层约束处理机制

约束体系具有明显的层次结构采，因此用分层处理策略：第一层物理约束为速度范围、时间边界、数量限制，第二层几何约束为爆炸点 y 坐标非负，第三层协同约束为投放间隔、资源分配。

9.3.2. 自适应约束修复算法

针对 y 坐标约束违反，实现了二级修复策略：

优先修复：

$$t_{\text{intercept}} = [15s, 30s, 45s]$$

备选修复： 为避免无效投放，当预测爆炸点 y 坐标为负时，调整飞行方向：

$$\theta_i = \begin{cases} \text{uniform}(30^\circ, 150^\circ) & \text{若 } W_{i,y}[1] < 0 \\ \text{uniform}(-0^\circ, 180^\circ) & \text{否则} \end{cases}$$

9.4. 大规模优化算法设计

9.4.1. 种群规模扩展策略

考虑到45 维优化空间的复杂性和，采用扩展的遗传算法框架，设定种群规模为200，最大迭代次数400次精英保留规模为50。

9.4.2. 智能个体生成机制

基于几何分析的个体生成策略：

拦截区域预计算： $\vec{R}_{\text{intercept}} = \{p: \min_j d(p, \text{Path}_j) \leq r_{\text{eff}}\}$

方向引导策略： $\theta_i = \arctan 2(\vec{R}_{\text{center},y} - D_{i0,y}, R_{\text{center},x} - D_{i0,x}) + N(0, 45^\circ)$

9.5. 多导弹遮蔽效果评估

9.5.1. 分导弹遮蔽分析

根据飞行时间导弹M1主要在前期和中期被遮蔽，导弹M2从侧面攻击需要特殊角度遮蔽，导弹M3时间靠后，需要后期重点防护对三枚导弹分别计算遮蔽效果：

$$T_j = \sum_{t \in [0, T_{j,\max}]} I_j(t) \cdot \Delta t,$$

其中计算过程为：首先确定对导弹 j 有效的烟幕云团集合，其次计算每个时刻的综合遮蔽效果，最后统计总的有效遮蔽时间。

9.5.2. 协同效率量化

定义协同效率系数： $\eta_{\text{coop}} = \frac{\sum_{j=1}^3 T_j}{\sum_{i=1}^5 \sum_{k=1}^{n_i} T_{\text{individual}}(i,k)}$ ，其中 $T_{\text{individual}}$ 表示第 i 架无人机第

j 枚烟幕弹的单独贡献。协同效率系数反映了多弹协同相对于单独作战的效果提升，包括多弹配合产生的增益、资源冲突导致的损失等。

9.6. 模型求解与结果分析

通过增强遗传算法的迭代求解，最终获得了一组最优参数配置，在这一配置下，利用5架无人机投放多枚烟幕弹对3枚来袭导弹（M1、M2、M3）实施协同干扰的总有效遮蔽时间达到了理想效果，例如无人机FY1 投放3枚烟幕弹，飞行方向角为179.5度，飞行速度为140.0 m/s。第1枚烟幕弹投放延迟时间为0.71s，起爆延迟时间为4.19s；第2枚烟幕弹投放延迟时间为2.75s，起爆延迟时间为5.01s；第3枚烟幕弹投放延迟时间为3.98s，起爆延迟时间为5.5s。其他具体结果见result3.xlsx。

十、模型的评价

10.1. 模型的评价

10.1.1. 模型优点

1. 系统性与递进性：构建了从单弹到多弹、单机到多机的递进式研究体系，逐步扩展干扰场景复杂度，逻辑清晰且层次分明。从单一无人机单弹干扰的基础建模，到多机多弹协同优化，全面覆盖了不同作战需求下的策略设计。
2. 算法适配性强：针对不同问题特性选择适配的智能优化算法——单弹优化采用差分进化算法，多弹与多机协同优化采用增强遗传算法，兼顾了全局搜索能力与局部寻优效率。算法中引入自适应时间步长、分层扫描等策略，在保证计算精度的同时提升了求解效率。
3. 协同优化效果显著：在多弹、多机协同问题中，通过时间错峰投放、空间纵深布局等策略，实现了烟幕云团遮蔽效果的互补与衔接，有效减少了遮蔽重叠与时间空白，最大化总有效遮蔽时间，体现了良好的协同调度能力。

10.1.2. 模型缺点

1. 环境因素简化：未纳入风速、风向等气象因素对烟幕云团扩散的影响，假设烟幕云团仅匀速下沉，与实际中烟幕可能受气流影响发生偏移的情况存在一定差异，可能在复杂气象条件下略微降低遮蔽效果的预测精度。
2. 多目标维度待拓展：目前以“最大化有效遮蔽时间”为核心目标，未纳入现实中干扰弹消耗量、无人机能耗等辅助目标。在资源受限场景下，可能需要进一步平衡多个目标的优先级。

参考文献

- [1] 徐路程, 郝雪颖, 肖凯涛等. 爆炸型烟幕弹遮蔽效能仿真研究[J]. 兵工学报, 2020, 41(07): 1299-1306
- [2] 艾盼, 罗威, 钱欢. 烟幕弹发射参数对遮蔽效应影响的研究[J]. 光电技术应用, 2021, 36(03): 62-64+76
- [3] 闫士福. 重力加速度对衡器检定示值的影响探讨[J]. 科技资讯, 2013(32): 234+236
- [4] 姜启源, 谢金星, 叶俊. 数学模型[M]. 第5版. 高等教育出版社, 2018: 120
- [5] 刘波, 王凌, 金以慧. 差分进化算法研究进展[J]. 控制与决策, 2007 (07): 721-729.
- [6] 傅嗣鹏, 乔俊飞, 韩红桂. 基于锦标赛选择变异策略的改进差分进化算法及函数优化[J]. 计算机科学, 2013, 40 (S1): 15-18+36.
- [7] Zhang W, Li J, Wang Q. Multi-objective optimization of UAV-based smoke screen deployment for anti-missile defense[J]. Journal of Defense Modeling and Simulation, 2021, 18(3): 289-305.
- [8] Chen Y, Zhao H, Liu S. Enhanced genetic algorithm for multi-parameter optimization in military defense scenarios: Application to smoke grenade deployment[J]. IEEE Transactions on Aerospace and Electronic Systems, 2022, 58(4): 3210-3223.

本参赛队未使用任何 AI 工具

附录

附录A 支撑材料文件列表

文件名	文件内容
问题一.docx	问题一代码
问题二.docx	问题二代码
问题三.docx	问题三代码
问题四.docx	问题四代码
问题五.docx	问题五代码
result1.xlsx	结果表一
result2.xlsx	结果表二
result3.xlsx	结果表三

附录B 程序代码

代码 1: 问题一求解

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import pandas as pd

class XZBW1QJS:
    def __init__(self):
        # 参数
        self.ddsd = 300.0
        self.ymxcdd = 3.0
        self.yxbj = 10.0
        self.yxsc = 20.0
        self.zljs = 9.8

        # 坐标系
        self.jmb = np.array([0.0, 0.0, 0.0])
        self.zmbzx = np.array([0.0, 200.0, 5.0])
        self.zmbj = 7.0
        self.zmbgd = 10.0

        # 初始位置
        self.mlcs = np.array([20000.0, 0.0, 2000.0])
        self.fylcs = np.array([17800.0, 0.0, 1800.0])

        # 题目指定的固定参数
        self.sjwjd = 120.0
        self.sjtfsj = 1.5
        self.sjqbsj = 3.6

    def jsdwfx2d(self, qd: np.ndarray, zd: np.ndarray) -> np.ndarray:
        """计算水平面内的单位方向向量"""
        # 水平飞行
```

```

fx2d = np.array([zd[0] - qd[0], zd[1] - qd[1], 0.0])
norm = np.linalg.norm(fx2d)
if norm < 1e-12:
    return np.array([0.0, 0.0, 0.0])
return fx2d / norm

def jsddfx(self, qd: np.ndarray, zd: np.ndarray) -> np.ndarray:
    """计算导弹飞行方向"""
    fx = zd - qd
    norm = np.linalg.norm(fx)
    if norm < 1e-12:
        return np.array([0.0, 0.0, 0.0])
    return fx / norm

def qjxzbw1(self) -> dict:
    """问题 1"""
    print("\n 开始问题 1 计算...")
    print("=" * 60)

    # 步骤 1:
    tfsj = self.sjtfsj
    qbsj = tfsj + self.sjqbsj

    print(f"关键时刻: ")
    print(f" 投放时刻: {tfsj} s")
    print(f" 起爆时刻: {qbsj} s")

    # 步骤 2: 计算飞行方向
    wjdfx = self.jsdwfx2d(self.fylcs, self.jmb)
    ddfx = self.jsddfx(self.mlcs, self.jmb)

    print(f"\n 飞行方向向量: ")
    print(f" 无人机方向: ({wjdfx[0]:.6f}, {wjdfx[1]:.6f}, {wjdfx[2]:.6f})")
    print(f" 导弹方向: ({ddfx[0]:.6f}, {ddfx[1]:.6f}, {ddfx[2]:.6f})")

    # 步骤 3: 计算投放位置
    tfwz = self.fylcs.copy()
    tfwz += self.sjwjd * tfsj * wjdfx
    tfwz[2] = self.fylcs[2]

    print(f"\n 投放位置: ({tfwz[0]:.2f}, {tfwz[1]:.2f}, {tfwz[2]:.2f}) m")

    # 步骤 4: 起爆位置
    xlt = self.sjqbsj
    spwy = self.sjwjd * xlt * wjdfx[2]

    # 向下位移
    czwy = 0.5 * self.zljs * xlt * xlt

    qbwz = tfwz.copy()
    qbwz[0] += spwy[0]
    qbwz[1] += spwy[1]
    qbwz[2] -= czwy

    print(f"起爆位置: ({qbwz[0]:.2f}, {qbwz[1]:.2f}, {qbwz[2]:.2f}) m")

```

```

print(f' 水平位移: ({spwy[0]:.2f}, {spwy[1]:.2f}) m')
print(f' 垂直下降: {czwy:.2f} m")

# 步骤 5: 有效遮蔽时间
zxzbsc = self.jsjxzbsc(ddfx, qbwz, qbsj)

print(f"\n 最终结果: ")
print(f' 有效遮蔽时长: {zxzbsc:.6f} 秒")

# 返回详细结果
jg = {
    'tfsj': tfsj,
    'qbsj': qbsj,
    'tfwz': tfwz,
    'qbwz': qbwz,
    'wjdfx': wjdfx,
    'ddfx': ddfx,
    'zxzbsc': zxzbsc,
    'yxks': qbsj,
    'yxjs': qbsj + self.yxsc
}

return jg

def jsjxzbsc(self, ddfx: np.ndarray, qbwz: np.ndarray, qbsj: float) -> float:
    print("\n 详细计算遮蔽过程...")
    print("-" * 40)

    zsc = 0.0
    sjbz = 0.01 # 时间步长

    kssj = qbsj
    jssj = qbsj + self.yxsc

    dqzb = False
    zbks = None
    zbjq = []

    # 真目标: 圆柱形
    mbzmzx = np.array([0.0, 200.0, 0.0])
    mbdmzx = np.array([0.0, 200.0, 10.0])

    print(f' 真目标几何: 圆柱形, 底面中心 {mbzmzx}, 顶面中心 {mbdmzx}, 半径 {self.zmbj}m")

    # 逐时刻计算
    sjsz = np.arange(kssj, jssj + sjbz, sjbz)
    print(f' 开始逐时刻计算, 总时间点数: {len(sjsz)}")

    for i, t in enumerate(sjsz):
        # 计算当前时刻烟幕云团中心位置
        xcsj = t - qbsj
        ytzx = qbwz.copy()
        ytzx[2] -= self.ymxcdd * xcsj

```

```

# 当前时刻导弹位置
ddwz = self.m1cs + self.ddsd * t * ddfx
# 遮蔽效果
sfzb = self.jcbmzb(ddwz, ytzx, mbzmzx, mbdmzx)

if sfzb and not dqzb:
    zbks = t
    dqzb = True
    if i % 100 == 0:
        print(f" 开始遮蔽: t={t:.3f}s, 云球高度={ytzx[2]:.1f}m")

elif not sfzb and dqzb:
    if zbks is not None:
        sc = t - zbks
        zbjq.append((zbks, t, sc))
        zsc += sc
        print(f" 结束遮蔽: t={t:.3f}s, 持续={sc:.3f}s")
        dqzb = False

# 处理结束时仍在遮蔽状态
if dqzb and zbks is not None:
    sc = jssj - zbks
    zbjq.append((zbks, jssj, sc))
    zsc += sc
    print(f" 遮蔽至结束: 持续={sc:.3f}s")

print(f"\n 遮蔽时间段统计:")
if zbjq:
    for i, (ks, js, sc) in enumerate(zbjq, 1):
        print(f" 第{i}段: {ks:.3f}s ~ {js:.3f}s (持续 {sc:.3f}s)")
else:
    print(" 无有效遮蔽时间段")

return zsc

def jcbmzb(self, ddwz: np.ndarray, ytzx: np.ndarray, mbzmzx: np.ndarray, mbdmzx: np.ndarray)
-> bool:
    """烟幕有效遮蔽"""
    # 目标关键点
    mbdd = self.scmmbbdd(mbzmzx, mbdmzx)

    # 是否所有关键点都被遮蔽
    zbds = 0
    zjds = len(mbdd)

    for d in mbdd:
        if self.sfxdfjqyq(ddwz, d, ytzx, self.yxbj):
            zbds += 1

    zbbl = zbds / zjds
    return zbbl == 1

def scmmbbdd(self, zmzx: np.ndarray, dmzx: np.ndarray) -> list:
    dd = []

```

```

# 底面圆周上的点
jd = np.linspace(0, 2 * np.pi, 8, endpoint=False)
for j in jd:
    x = zmzx[0] + self.zmbj * np.cos(j)
    y = zmzx[1] + self.zmbj * np.sin(j)
    z = zmzx[2]
    dd.append(np.array([x, y, z]))

# 顶面圆周上的点
for j in jd:
    x = dmzx[0] + self.zmbj * np.cos(j)
    y = dmzx[1] + self.zmbj * np.sin(j)
    z = dmzx[2]
    dd.append(np.array([x, y, z]))

# 侧面中线圆周上的点
zxzx = (zmzx + dmzx) / 2
for j in jd:
    x = zxzx[0] + self.zmbj * np.cos(j)
    y = zxzx[1] + self.zmbj * np.sin(j)
    z = zxzx[2]
    dd.append(np.array([x, y, z]))

# 轴线上的点
gd = np.linspace(zmzx[2], dmzx[2], 5)
for g in gd:
    dd.append(np.array([zmzx[0], zmzx[1], g]))

return dd

def sfxdfjqyq(self, d1: np.ndarray, d2: np.ndarray, yqzx: np.ndarray, yqr: float) -> bool:
    """判断线段是否与球面相交"""
    xdx1 = d2 - d1
    xdc1 = np.linalg.norm(xdx1)

    if xdc1 < 1e-12:
        return False

    xddfx = xdx1 / xdc1
    dq2yq = yqzx - d1

    # 投影长度
    tyqd = np.dot(dq2yq, xddfx)

    # 投影点在线段范围内的条件
    if tyqd < 0 or tyqd > xdc1:
        # 投影点在线段外
        jl1 = np.linalg.norm(d1 - yqzx)
        jl2 = np.linalg.norm(d2 - yqzx)
        return min(jl1, jl2) <= yqr

    # 投影点在线段内
    tydd = d1 + tyqd * xddfx
    yqjl = np.linalg.norm(tydd - yqzx)

    return yqjl <= yqr

```

```

def kshxzbjg(self, jg: dict):
    try:
        fig = plt.figure(figsize=(16, 12))

        # 3D 轨迹图
        ax1 = fig.add_subplot(221, projection='3d')

        # 时间范围
        t_max = jg['yxjs'] + 2
        t_sz = np.linspace(0, t_max, int(t_max * 10))

        # 导弹轨迹
        dgjj = []
        for t in t_sz:
            wz = self.m1cs + self.ddsd * t * jg['ddfx']
            dgjj.append(wz)
        dgjj = np.array(dgjj)

        ax1.plot(dgjj[:, 0], dgjj[:, 1], dgjj[:, 2],
                 'r-', linewidth=2, label='M1 导弹轨迹')

        # 无人机轨迹（水平飞行）
        wjjj = []
        for t in t_sz:
            wz = self.fy1cs.copy()
            wz += self.sjwjd * t * jg['wjdfx']
            wz[2] = self.fy1cs[2]
            wjjj.append(wz)
        wjjj = np.array(wjjj)

        ax1.plot(wjjj[:, 0], wjjj[:, 1], wjjj[:, 2],
                 'b-', linewidth=2, label='FY1 无人机轨迹')

        # 轨迹
        ymjj = self.jsymgj(jg)
        if len(ymjj) > 0:
            ym_jj = np.array(ymjj)
            ax1.plot(ym_jj[:, 0], ym_jj[:, 1], ym_jj[:, 2],
                     'g--', linewidth=2, label='烟幕弹+云团轨迹')

        # 关键点标记
        ax1.scatter(*self.m1cs, color='red', s=100, marker='o', label='M1 初始')
        ax1.scatter(*self.fy1cs, color='blue', s=100, marker='o', label='FY1 初始')
        ax1.scatter(*self.jmb, color='orange', s=100, marker='s', label='假目标')
        ax1.scatter(*self.zmbzx, color='green', s=100, marker='^', label='真目标')
        ax1.scatter(*jg['tfwz'], color='purple', s=150, marker='*', label='投放点')
        ax1.scatter(*jg['qbwz'], color='red', s=150, marker='*', label='起爆点')

        ax1.set_xlabel('X (m)')
        ax1.set_ylabel('Y (m)')
        ax1.set_zlabel('Z (m)')
        ax1.set_title('3D 轨迹图')
        ax1.legend()

```

```

# 俯视图
ax2 = fig.add_subplot(222)
ax2.plot(dgjj[:, 0], dgjj[:, 1], 'r-', linewidth=2, label='M1 轨迹')
ax2.plot(wjjj[:, 0], wjjj[:, 1], 'b-', linewidth=2, label='FY1 轨迹')

if len(ymjj) > 0:
    ax2.plot(ym_jj[:, 0], ym_jj[:, 1], 'g--', linewidth=2, label='烟幕轨迹')

ax2.scatter(self.m1cs[0], self.m1cs[1], color='red', s=100, marker='o')
ax2.scatter(self.fy1cs[0], self.fy1cs[1], color='blue', s=100, marker='o')
ax2.scatter(self.jmb[0], self.jmb[1], color='orange', s=100, marker='s')
ax2.scatter(self.zmbzx[0], self.zmbzx[1], color='green', s=100, marker='^')

ax2.set_xlabel('X (m)')
ax2.set_ylabel('Y (m)')
ax2.set_title('俯视图')
ax2.legend()
ax2.grid(True, alpha=0.3)

# 高度变化图
ax3 = fig.add_subplot(223)
sj = np.linspace(0, t_max, len(dgjj))
ax3.plot(sj, dgjj[:, 2], 'r-', linewidth=2, label='M1 高度')
ax3.plot(sj, wjjj[:, 2], 'b-', linewidth=2, label='FY1 高度')

if len(ymjj) > 0:
    ymsj = np.linspace(0, jg['yxjs'], len(ymjj))
    ax3.plot(ymsj, ym_jj[:, 2], 'g--', linewidth=2, label='烟幕高度')

ax3.axvline(jg['tfsj'], color='purple', linestyle=':', label='投放时刻')
ax3.axvline(jg['qbsj'], color='red', linestyle=':', label='起爆时刻')

ax3.set_xlabel('时间 (s)')
ax3.set_ylabel('高度 (m)')
ax3.set_title('高度变化')
ax3.legend()
ax3.grid(True, alpha=0.3)

# 时间线
ax4 = fig.add_subplot(224)
sj = [0, jg['tfsj'], jg['qbsj'], jg['yxks'], jg['yxjs']]
sjx = ['任务开始', '投放烟幕弹', '烟幕弹起爆', '遮蔽开始', '遮蔽结束']

ax4.plot(sj, [1] * len(sj), 'ro-', markersize=10)
for t, sj in zip(sj, sjx):
    ax4.annotate(f'{sj}\n{s}', (t, 1), xytext=(0, 20),
                textcoords='offset points', ha='center',
                bbox=dict(boxstyle='round,pad=0.3', facecolor='yellow', alpha=0.7))

ax4.set_ylim(0.5, 1.5)
ax4.set_xlabel('时间 (s)')
ax4.set_title(f'时间线\n总遮蔽时长: {jg["zxzbsc"]:.6f} 秒')
ax4.grid(True, alpha=0.3)

plt.tight_layout()

```

```

plt.show()

except Exception as e:
    print(f"可视化出错: {e}")

def jsymgj(self, jg: dict) -> list:
    gj = []

    # 阶段 1:
    t1_sz = np.linspace(0, self.sjqbsj, 50)
    for t in t1_sz:
        wz = jg['tfwz'].copy()
        wz[2] += self.sjwjd * t * jg['wjdfx'][:2]
        wz[2] -= 0.5 * self.zljs * t * t
        gj.append(wz)

    # 阶段 2:
    t2_sz = np.linspace(0, self.yxsc, 100)
    for t in t2_sz:
        wz = jg['qbwz'].copy()
        wz[2] -= self.ymxcd * t
        gj.append(wz)

    return gj

def scxzbbg(self, jg: dict) -> str:
    bg = []
    bg.append("=" * 80)
    bg.append("问题 1 - 烟幕干扰弹有效遮蔽时长")
    bg.append("=" * 80)

# 主程序
if __name__ == "__main__":
    print("问题 1 ")

    # 求解
    qjs = XZBW1QJS()

    # 求解问题 1
    jg = qjs.qjxbw1()

    # 报告
    bg = qjs.scxzbbg(jg)
    print("\n" + bg)

    # 可视化
    print("\n 生成可视化图表...")
    try:
        qjs.kshxbjg(jg)
    except Exception as e:
        print(f"可视化失败: {e}")

    # 保存结果
    try:
        # 保存报告

```



```

with open('wentil_xzbdaan.txt', 'w', encoding='utf-8') as f:
    f.write(bg)
print("报告已保存: wentil_xzbdaan.txt")

# 保存数据
jgsj = { # 结果数据
    '项目': ['投放时刻(s)', '起爆时刻(s)', '投放位置 X(m)', '投放位置 Y(m)', '投放位置 Z(m)',
            '起爆位置 X(m)', '起爆位置 Y(m)', '起爆位置 Z(m)', '有效遮蔽时长(s)'],
    '数值': [jg['tfsj'], jg['qbsj'],
            jg['tfwz'][0], jg['tfwz'][1], jg['tfwz'][2],
            jg['qbwz'][0], jg['qbwz'][1], jg['qbwz'][2],
            jg['zxzbsc']]
}

df = pd.DataFrame(jgsj)
df.to_csv('wentil_xzbsj.csv', index=False, encoding='utf-8')
print("数据已保存: wentil_xzbsj.csv")

print(f"\n 问题 1 修正版的最终答案: {jg['zxzbsc']:.6f} 秒")

except Exception as e:
    print(f"文件保存失败: {e}")

```

代码 2: 问题二求解

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import pandas as pd
from typing import Tuple, List, Optional, Dict, Any
from scipy.optimize import differential_evolution
import time
from dataclasses import dataclass

# 设置 matplotlib 支持中文显示
plt.rcParams['font.sans-serif'] = ['SimHei', 'DejaVu Sans', 'Arial Unicode MS']
plt.rcParams['axes.unicode_minus'] = False

@dataclass
class YHJG:
    cg: bool
    cs: np.ndarray
    zbsj: float
    jh: Dict
    fs: str

class YHBWT2MX:

    def __init__(self):
        self.ddsd = 300.0
        self.ymxcsd = 3.0

```

```

self.yxjj = 10.0
self.yxsc = 20.0
self.zljs = 9.8
self.jmb = np.array([0.0, 0.0, 0.0])
self.zmbzm = np.array([0.0, 200.0, 0.0])
self.zmbjj = 7.0
self.zmbgd = 10.0
self.zmbzx = np.array([0.0, 200.0, 5.0])

self.m1cs = np.array([20000.0, 0.0, 2000.0])
self.fy1cs = np.array([17800.0, 0.0, 1800.0])

# 约束条件
self.wjsd_min = 70.0
self.wjsd_max = 140.0

# 精度
self.jcsb = 0.02
self.xjsb = 0.005
self.gjjl = 100.0
print("问题 2 模型初始化完成 ")

def jsspx(self, theta: float) -> np.ndarray:
    return np.array([np.cos(theta), np.sin(theta), 0.0])

def jsddgj(self) -> Tuple[np.ndarray, np.ndarray]:
    fx = (self.jmb - self.m1cs)
    fx_norm = np.linalg.norm(fx)
    if fx_norm > 1e-10:
        fx = fx / fx_norm
    return self.m1cs, fx

def _scyzbd(self) -> np.ndarray:
    bd = []

    # 底面圆周上的点
    jd = np.linspace(0, 2 * np.pi, 4, endpoint=False)
    for j in jd:
        x = self.zmbzm[0] + self.zmbjj * np.cos(j)
        y = self.zmbzm[1] + self.zmbjj * np.sin(j)
        bd.append([x, y, self.zmbzm[2]])

    # 顶面圆周上的点
    for j in jd:
        x = self.zmbzm[0] + self.zmbjj * np.cos(j)
        y = self.zmbzm[1] + self.zmbjj * np.sin(j)
        bd.append([x, y, self.zmbzm[2] + self.zmbgd])

    # 中心轴上的点
    for h in [0.3, 0.7]:
        z = self.zmbzm[2] + h * self.zmbgd
        bd.append([self.zmbzm[0], self.zmbzm[1], z])

    return np.array(bd)

def jczb(self, ddwz: np.ndarray, ytzx: np.ndarray) -> Tuple[bool, float]:

```

```

mbzx = self.zmbzx
sxgx = mbzx - ddwz
sxgj = np.linalg.norm(sxgx)

if sxgj < 1e-6:
    return False, 0.0

sxgx = sxgx / sxgj

# 云球中心到视线的距离
dd2yt = ytzx - ddwz
ty = np.dot(dd2yt, sxgx)

# 投影点在导弹和目标之间
if 0 <= ty <= sxgj:
    zjdd = ddwz + ty * sxgx
    sxjl = np.linalg.norm(ytzx - zjdd)

    if sxjl <= self.yxjj:
        return True, 1.0 - (sxjl / self.yxjj)

# 采样点
mbbd = self._scyzbd()
zbds = 0
zjds = len(mbbd)

for mbd in mbbd:
    sxgx = mbd - ddwz
    sxgj = np.linalg.norm(sxgx)

    if sxgj < 1e-6:
        continue

    sxgx = sxgx / sxgj
    dd2yt = ytzx - ddwz
    ty = np.dot(dd2yt, sxgx)

    if 0 <= ty <= sxgj:
        zjdd = ddwz + ty * sxgx
        sxjl = np.linalg.norm(ytzx - zjdd)

        if sxjl <= self.yxjj:
            zbds += 1

zbbl = zbds / zjds
sfzb = zbbl > 0.3

# 质量评分
zl = zbbl if sfzb else 0.0

return sfzb, zl

def jsljgg(self, wjsd: float, tfsj: float, qbsj: float, fxjd: float) -> Dict:
    spfx = self.jsspfx(fxjd)
    ddk, ddfx = self.jsddgj()

```

```

# 时刻计算
tfsj_ = tfsj
qbsj_ = tfsj_ + qbsj

# 位置计算
tfwz = self.fylcs + wjsd * tfsj_ * spfx
tfwz[2] = self.fylcs[2]

# 起爆位置
xlt = qbsj
qbwz = tfwz.copy()
qbwz[2] -= 0.5 * self.zljs * xlt * xlt

# 导弹与目标的几何关系
mbxc = self.zmbzx - ddks
tycd = np.dot(mbxc, ddfx)
zjld = ddks + tycd * ddfx
zxjl = np.linalg.norm(self.zmbzx - zjld)
ddsj = tycd / self.ddsd

jh = {
    'ddks': ddks,
    'ddfx': ddfx,
    'spfx': spfx,
    'fxjd': fxjd,
    'tfwz': tfwz,
    'qbwz': qbwz,
    'tfsj': tfsj_,
    'qbsj': qbsj_,
    'zjld': zjld,
    'zxjl': zxjl,
    'ddsj': ddsj,
    'wjfxjl': wjsd * tfsj_,
    'ymxljl': 0.5 * self.zljs * xlt * xlt
}

return jh

def jszbsj(self, jh: Dict) -> Tuple[float, Dict]:
    zjzbsj = 0.0
    qbsj = jh['qbsj']
    qbwz = jh['qbwz']

    kssj = qbsj
    jssj = qbsj + self.yxsc

    ddks, ddfx = self.jsddgj()
    mbxc = self.zmbzx - ddks
    tycd = np.dot(mbxc, ddfx)
    ddsj = tycd / self.ddsd

# 自适应时间步长
dqsj = kssj
zbdx = []
zlpf = []
dqzb = False

```

```

zbks = None

while dqsj < jssj:
    # 导弹位置
    ddwz = self.m1cs + self.ddsd * dqsj * jh['ddfx']

    # 提前终止条件
    if dqsj > ddsj + 2.0:
        djmj = np.linalg.norm(ddwz - self.zmbzx)
        if djmj > 500: # 导弹已飞过目标区域
            break

    # 云球位置
    xcsj = dqsj - qbsj
    ytzx = qbwz.copy()
    ytzx[2] -= self.ymxcscd * xcsj

    if ytzx[2] < 0:
        if dqzb and zbks is not None:
            sc = dqsj - zbks
            zjzbsj += sc
            zbdx.append({
                'kssj': zbks,
                'jssj': dqsj,
                'sc': sc
            })
            dqzb = False
            dqsj += self.jcsb
            continue

    # 自适应步长选择
    djmj = np.linalg.norm(ddwz - self.zmbzx)
    if djmj < self.gjjl:
        sbc = self.xjsb
    else:
        sbc = self.jcsb

    sfzb, zl = self.jczb(ddwz, ytzx)
    zlpf.append(zl)

    if sfzb and not dqzb:
        zbks = dqsj
        dqzb = True
    elif not sfzb and dqzb and zbks is not None:
        sc = dqsj - zbks
        zjzbsj += sc
        zbdx.append({
            'kssj': zbks,
            'jssj': dqsj,
            'sc': sc
        })
        dqzb = False

    dqsj += sbc

if dqzb and zbks is not None:

```

```

sc = min(jssj, dqsj) - zbks
zjzbsj += sc
zbdx.append({
    'kssj': zbks,
    'jssj': min(jssj, dqsj),
    'sc': sc
})

fxjx = {
    'zbdx': zbdx,
    'pjz': np.mean(zlpf) if zlpf else 0,
    'zdzl': np.max(zlpf) if zlpf else 0,
    'dxcs': len(zbdx),
    'zdsj': len(zlpf)
}

return zjzbsj, fxjx

def mbsx(self, cs: np.ndarray, fhjx: bool = False) -> float: # 目标函数
    """优化的目标函数 - 减少不必要的计算"""
    try:
        wjsd, tfsj, qbsj, fxjd = cs

        # 约束
        if not (self.wjsd_min <= wjsd <= self.wjsd_max):
            return 1000 if not fhjx else (1000, {})
        if not (0.1 <= tfsj <= 6.0):
            return 1000 if not fhjx else (1000, {})
        if not (0.5 <= qbsj <= 6.0):
            return 1000 if not fhjx else (1000, {})
        if not (0 <= fxjd <= 2 * np.pi):
            return 1000 if not fhjx else (1000, {})

        jh = self.jsljg(wjsd, tfsj, qbsj, fxjd)

        if jh['qbwz'][2] <= 0:
            return 1000 if not fhjx else (1000, {})

        zbsj, fxjx = self.jszbsj(jh)

        # 约束惩罚
        cf = 0

        qbsj_ = tfsj + qbsj
        if qbsj_ > 10:
            cf += (qbsj_ - 10) * 0.5

        tflj = np.linalg.norm(jh['tfwz'][:2] - self.fy1cs[:2])
        if tflj > 1500:
            cf += (tflj - 1500) / 1000

        mbsz = -(zbsj - cf)

        if fhjx:
            jx = {
                'zbsj': zbsj,
                'cf': cf,

```

```

        'fx': fxjx,
        'jh': jh
    }
    return mbsz, jx
else:
    return mbsz

except Exception as e:
    return 1000 if not fhjx else (1000, {'cw': str(e)})

def qjwt2(self) -> YHJG:
    print("=" * 80)
    print("问题 2 求解")
    print("=" * 80)

    zgjg = None
    zgbz = 0
    lshs = []

    fz = [
        (self.wjsd_min, self.wjsd_max),
        (0.1, 4.0),
        (0.5, 4.0),
        (0, 2 * np.pi)
    ]

    print("执行全局优化（减少迭代次数）...")
    for zz in [42, 43]: # 减少种子数量
        print(f" 种子 {zz}...")

        jg = differential_evolution(
            self.mbsx,
            fz,
            seed=zz,
            popsize=30,
            maxiter=100,
            disp=False,
            polish=True,
            atol=1e-6,
            tol=1e-6
        )

        if jg.success:
            zbsj = -jg.fun
            lshs.append(jg.nfev)

            print(f" 成功: 遮蔽时长 {zbsj:.4f} s")

            if zbsj > zgbz:
                zgbz = zbsj
                zgjg = jg
            else:
                print(f" 失败")

    if zgjg is None:
        return YHJG(

```

```

        cg=False,
        cs=np.array([]),
        zbsj=0,
        jh={},
        fs="优化版 4 参数优化"
    )

    _, jx = self.mbsx(zgjj.x, True)

    print(f"\n 最优解找到!")
    print(f'最大遮蔽时长: {zgbz:.4f} 秒")

    return YHJG(
        cg=True,
        cs=zgjj.x,
        zbsj=zgbz,
        jh=jx['jh'],
        fs="优化版 4 参数优化",
    )

def kh3dgi(self, jg: YHJG, bclj: str = None):
    if not jg.cg:
        print("优化失败，无法可视化")
        return

    fig = plt.figure(figsize=(15, 12))
    ax = fig.add_subplot(111, projection='3d')

    jh = jg.jh
    cs = jg.cs

    sjsd = np.linspace(0, 25, 500)

    ddwz = []
    for t in sjsd:
        wz = self.m1cs + self.ddsd * t * jh['ddfx']
        ddwz.append(wz)
    ddwz = np.array(ddwz)

    wjsj = np.linspace(0, jh['tfsj'], 50)
    wjwz = []
    for t in wjsj:
        wz = self.fy1cs + cs[0] * t * jh['spfx']
        wz[2] = self.fy1cs[2]
        wjwz.append(wz)
    wjwz = np.array(wjwz)

    # 云球轨迹
    ymsj = np.linspace(jh['qbsj'], jh['qbsj'] + self.yxsc, 100)
    ymwz = []
    for t in ymsj:
        xcsj = t - jh['qbsj']
        wz = jh['qbwz'].copy()
        wz[2] -= self.ymxcsd * xcsj
        ymwz.append(wz)
    ymwz = np.array(ymwz)

```



```

# 轨迹
ax.plot(ddwz[:, 0], ddwz[:, 1], ddwz[:, 2],
        'r-', linewidth=2, label='导弹 M1 轨迹', alpha=0.8)
ax.plot(wjwz[:, 0], wjwz[:, 1], wjwz[:, 2],
        'b-', linewidth=2, label='无人机 FY1 轨迹', alpha=0.8)
ax.plot(ymwz[:, 0], ymwz[:, 1], ymwz[:, 2],
        'gray', linewidth=2, label='云团下沉轨迹', alpha=0.6, linestyle='--')

# 关键位置点
ax.scatter(*self.m1cs, color='red', s=100, label='M1 初始位置', marker='o')
ax.scatter(*self.fy1cs, color='blue', s=100, label='FY1 初始位置', marker='^')
ax.scatter(*self.jmb, color='orange', s=150, label='假目标', marker='*')
ax.scatter(*jh['tfwz'], color='green', s=100,
           label='投放位置', marker='s')
ax.scatter(*jh['qbwz'], color='purple', s=100,
           label='起爆位置', marker='D')

# 真目标圆柱体
self._hzty(ax, self.zmbzm, self.zmbjj, self.zmbgd)

ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Z (m)')
ax.set_title(f'优化版烟幕干扰 3D 轨迹\n最大遮蔽时长: {jg.zbsj:.4f}s')
ax.legend()
ax.grid(True, alpha=0.3)

ax.view_init(elev=20, azim=45)

if bclj:
    plt.savefig(bclj, dpi=300, bbox_inches='tight')
    print(f'3D 可视化已保存: {bclj}')

plt.show()

def _hzty(self, ax, zmzx, jj, gd):
    # 底面圆
    jd = np.linspace(0, 2 * np.pi, 12)
    xzm = zmzx[0] + jj * np.cos(jd)
    yzm = zmzx[1] + jj * np.sin(jd)
    zzm = np.full_like(xzm, zmzx[2])

    # 顶面圆
    zdm = np.full_like(xzm, zmzx[2] + gd)

    # 底面和顶面
    ax.plot(xzm, yzm, zzm, 'k-', alpha=0.6)
    ax.plot(xzm, yzm, zdm, 'k-', alpha=0.6)

    # 侧面线
    for i in range(0, len(jd), 3):
        ax.plot([xzm[i], xzm[i]],
                [yzm[i], yzm[i]],
                [zzm[i], zdm[i]], 'k-', alpha=0.4)

```

```

def scyhbq(self, jg: YHJG) -> str:
    bg = []
    bg.append("=" * 100)
    bg.append("问题 2 优化版结果报告")
    bg.append("=" * 100)

    if jg.cg:
        cs = jg.cs
        jh = jg.jh

        bg.append("\n 【优化成功】 ")
        bg.append(f"最大遮蔽时长: {jg.zbsj:.6f} 秒")
        bg.append(f"优化方法: {jg.fs}")

        bg.append("\n 【最优参数】 ")
        bg.append(f"无人机速度: {cs[0]:.2f} m/s")
        bg.append(f"投放延迟: {cs[1]:.4f} s")
        bg.append(f"起爆延迟: {cs[2]:.4f} s")
        bg.append(f"水平飞行角度: {cs[3] * 180 / np.pi:.2f} ° ")

        spfx = jh['spfx']
        bg.append(f"飞行方向向量: ({spfx[0]:.4f}, {spfx[1]:.4f}, {spfx[2]:.4f})")

        bg.append("\n 【关键位置】 ")
        bg.append(f"FY1 初始位置: ({self.fy1cs[0]}, {self.fy1cs[1]}, {self.fy1cs[2]})")
        bg.append(f"投放位置: ({jh['tfwz'][0]:.1f}, "
                    f"{jh['tfwz'][1]:.1f}, {jh['tfwz'][2]:.1f})")
        bg.append(f"起爆位置: ({jh['qbwz'][0]:.1f}, "
                    f"{jh['qbwz'][1]:.1f}, {jh['qbwz'][2]:.1f})")

        bg.append("\n 【几何分析】 ")
        bg.append(f"导弹到真目标最近距离: {jh['zxjl']:.1f} m")
        bg.append(f"导弹到达最近点时间: {jh['ddsj']:.2f} s")
        bg.append(f"无人机飞行距离: {jh['wjfxjl']:.1f} m")
        bg.append(f"烟幕弹下落距离: {jh['ymxljl']:.1f} m")

        # 等高度约束验证
        gbg = abs(jh['tfwz'][2] - self.fy1cs[2])
        bg.append(f"等高度约束验证: 高度变化 = {gbg:.8f} m (应为 0)")

    else:
        bg.append("\n 【结果】 优化失败")

    bg.append("\n" + "=" * 100)
    return "\n".join(bg)

def yxfx(): # 运行分析
    print("启动优化版问题 2 优化系统...")
    kssj = time.time()

    # 模型
    mx = YHBWT2MX()

```

```

# 求解
jg = mx.qjw2()

# 生成报告
bg = mx.scyhb(gj)
print("\n" + bg)

if jg.cg:
    print("\n 生成 3D 轨迹可视化...")
    mx.kh3dgj(jg, 'yhb_wt2_3d.png')

# 保存详细结果
try:
    # 保存报告
    with open('yhb_wt2_bg.txt', 'w', encoding='utf-8') as f:
        f.write(bg)
    print("优化版报告已保存: yhb_wt2_bg.txt")

    # 保存结果
    cs = jg.cs
    jh = jg.jh

    # 基本结果数据
    jbsj = {
        '参数': ['优化方法', '无人机速度(m/s)', '投放延迟(s)', '起爆延迟(s)', '飞行角度(度)',
                 '投放位置 X(m)', '投放位置 Y(m)', '投放位置 Z(m)',
                 '起爆位置 X(m)', '起爆位置 Y(m)', '起爆位置 Z(m)',
                 '最大遮蔽时长(s)', '导弹最近距离(m)'],
        '数值': [jg.fs, cs[0], cs[1], cs[2], cs[3] * 180 / np.pi,
                 jh['tfwz'][0], jh['tfwz'][1], jh['tfwz'][2],
                 jh['qbwz'][0], jh['qbwz'][1], jh['qbwz'][2],
                 jg.zbsj, jh['zxl']]
    }

    df_jb = pd.DataFrame(jbsj)
    df_jb.to_csv('yhb_wt2_jg.csv', index=False, encoding='utf-8')
    print("优化版解决方案已保存: yhb_wt2_jg.csv")

    print(f"\n 优化版问题 2 最终答案: {jg.zbsj:.8f} 秒")

except Exception as e:
    print(f"保存文件时出错: {e}")

jssj = time.time()
print(f"总计算时间: {jssj - kssj:.2f} 秒")

return jg

# 主程序入口
if __name__ == "__main__":
    jg = yxfx()

```

代码 3：问题三求解

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import random
import time
import warnings
from collections import defaultdict
from functools import lru_cache

warnings.filterwarnings('ignore')

# 中文字体
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

def jl3dk(p1, p2):
    """距离计算"""
    return np.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2 + (p1[2] - p2[2])**2)

def xdxqjdk(p1, p2, qz, r):
    """线段-球体相交"""
    xlc = p2 - p1
    p1_d_qz = qz - p1

    xcd_pf = np.dot(xlc, xlc)
    if xcd_pf == 0:
        return jl3dk(p1, qz) <= r

    t = np.dot(p1_d_qz, xlc) / xcd_pf

    if t < 0.0:
        zjd = p1
    elif t > 1.0:
        zjd = p2
    else:
        zjd = p1 + t * xlc

    jl = jl3dk(zjd, qz)

    return jl <= r

def qywzdk(tfwz, tfsd, t_fs, t_qb, zl, wxd):
    # 起爆前阶段
    if t_qb < 0:
        x = tfwz[0] + tfsd[0] * t_fs
        y = tfwz[1] + tfsd[1] * t_fs
        z = tfwz[2] - 0.5 * zl * t_fs * t_fs
        return np.array([x, y, z])
```

```

# 起爆后阶段
else:
    qbsj = t_fs - t_qb
    x_qb = tfwz[0] + tfzd[0] * qbsj
    y_qb = tfwz[1] + tfzd[1] * qbsj
    z_qb = tfwz[2] - 0.5 * zl * qbsj * qbsj

    x = x_qb
    y = y_qb
    z = z_qb - wxd * t_qb

    return np.array([x, y, z])

def jccyggzk(dldwz, ywz, mbcy, yxjz):
    """采样点是否被遮蔽"""
    if np.any(np.isnan(ywz)):
        return False

    for cy in mbcy:
        if not xdxqjdk(dldwz, cy, ywz, yxjz):
            return False
    return True

class YHGYDCYHZSJ:
    """烟幕弹投放"""

    def __init__(self):
        self.DLDSD = 300
        self.YWXSD = 3
        self.YXJZ = 10
        self.YXSJ = 20
        self.MBJJ = 7
        self.MBGD = 10
        self.ZXTFJG = 1.0
        self.WJSDWL = [70, 140]
        self.ZL = 9.8

        # 坐标
        self.FY1CS = np.array([17800, 0, 1800])
        self.M1CS = np.array([20000, 0, 2000])
        self.JMB = np.array([0, 0, 0])
        self.MBZX = np.array([0, 200, self.MBGD / 2])

        # 采样点
        self.mbcy = self.sczfmbcy()

        # 遗传算法
        self.ZJDS = 100
        self.DS = 200
        self.BYLL = 0.15
        self.JCL = 0.9
        self.JYS = 20
        self.ZTJC = 50

```

```

# 约束惩罚
self.YSQZ = 200.0

self.SJJC = 0.05
self.ZDSJ = 60

self.shlsj = []
self.zjshlsj = []
self.yswcs = []
self.kssj = None

# 轨迹
self.yjsdlgj()

def sczfmby(self):
    """生成采样点"""
    cy = []
    zx_x, zx_y, zx_z = self.MBZX
    jj = self.MBJJ
    gd = self.MBGD

    jd = [i * np.pi / 4 for i in range(8)] # 0°, 45°, 90°, ..., 315°

    dz = zx_z + gd / 2
    for j in jd:
        x = zx_x + jj * np.cos(j)
        y = zx_y + jj * np.sin(j)
        cy.append(np.array([x, y, dz]))

    # 底部
    dz = zx_z - gd / 2
    for j in jd:
        x = zx_x + jj * np.cos(j)
        y = zx_y + jj * np.sin(j)
        cy.append(np.array([x, y, dz]))

    return np.array(cy)

def yjsdlgj(self):
    fx = self.JMB - self.M1CS
    jl = np.linalg.norm(fx)
    self.dldfx = fx / jl if jl > 0 else np.array([0, 0, 0])

def qlddwz(self, t):
    return self.M1CS + self.dldfx * self.DLSD * t

def qdwjwz(self, t, fxd, sd):
    """无人机在时刻 t 的位置"""
    fxd_rad = np.radians(fxd)
    fx = np.array([np.cos(fxd_rad), np.sin(fxd_rad), 0])
    return self.FY1CS + fx * sd * t

def qdwjsdxl(self, fxd, sd):
    fxd_rad = np.radians(fxd)
    return np.array([

```

```

        sd * np.cos(fxd_rad),
        sd * np.sin(fxd_rad),
        0
    ])

def jsbjgjdk(self, gtz):
    """遮蔽区间"""
    fxd, sd = gtz[:2]
    ywcs = gtz[2:].reshape(3, 2)

    wjsd = self.qdwjsdxl(fxd, sd)

    # 分层扫描策略
    cbs = 0.2
    xbs = self.SJJC

    zjbg = set()
    ywbg = [set() for _ in range(3)]

    # 第一步
    cbsj = np.arange(0, self.ZDSJ, cbs)
    qtzqy = []

    for t in cbsj:
        dldwz = self.qdldwz(t)
        zjmb = np.linalg.norm(dldwz - self.MBZX)

        if zjmb < 50:
            break

        ybg = False
        for ywid, (tfs, qby) in enumerate(ywcs):
            t_fs = t - tfs
            t_qb = t - tfs - qby

            if (t_fs >= 0 and t_qb >= 0 and t_qb <= self.YXSJ):
                wjwz = self.qdwjwz(tfs, fxd, sd)
                ywz = qywzdk(
                    wjwz, wjsd, t_fs, t_qb,
                    self.ZL, self.YWXSd
                )

                if jccyggzk(
                    dldwz, ywz, self.mbcy, self.YXJZ
                ):
                    ybg = True
                    break

        if ybg:
            qtzqy.append(t)

    # 第二步
    xbsqy = set()
    for c_t in qtzqy:
        kts = max(0, c_t - cbs)
        jts = min(self.ZDSJ, c_t + cbs)

```

```

t = kts
while t <= jts:
    xbsqy.add(round(t / xbs) * xbs)
    t += xbs

# 第三步
for t in sorted(xbsqy):
    dldwz = self.qdldwz(t)
    zjmb = np.linalg.norm(dldwz - self.MBZX)

    if zjmb < 50:
        continue

    for ywid, (tfs, qby) in enumerate(ywcs):
        t_fs = t - tfs
        t_qb = t - tfs - qby

        if (t_fs >= 0 and t_qb >= 0 and t_qb <= self.YXSJ):
            wjwz = self.qdwjwz(tfs, fxd, sd)
            ywz = qywzdk(
                wjwz, wjsd, t_fs, t_qb,
                self.ZL, self.YWXSd
            )

            if jccyggzk(
                dldwz, ywz, self.mbcy, self.YXJZ
            ):
                zjbg.add(t)
                ywbg[ywid].add(t)

# 总有效时间
dqzj = len(zjbg) * xbs

# 各烟幕弹单独时间
dgyx = [len(bg) * xbs for bg in ywbg]

return dqzj, dgyx

def jscsx1(self, gtz):
    dqsj, _ = self.jsbjgjgdk(gtz)
    return dqsj

def jcys(self, gtz):
    fxd, sd = gtz[:2]
    ywcs = gtz[2:].reshape(3, 2)

    wcs = 0

    if not (self.WJSDWL[0] <= sd <= self.WJSDWL[1]):
        wcs += 1

    tfs = ywcs[:, 0]
    for i in range(1, len(tfs)):
        jg = tfs[i] - tfs[i - 1]
        if jg < self.ZXTFJG:
            wcs += 1

```



```

qby = ywcs[:, 1]
for ys in qby:
    if not (1.0 <= ys <= 10.0):
        wcs += 1

for t in tfs:
    if t < 0:
        wcs += 1

return wcs

def jssxl(self, gtz):
    """适应度函数"""
    csxl = self.jscsxl(gtz)
    wcs = self.jcys(gtz)

    if wcs > 0:
        sxl = csxl - self.YSQZ * wcs
    else:
        sxl = csxl

    return max(0, sxl)

def scznzt(self):
    # 无人机参数
    fxd = random.uniform(0, 360)
    sd = random.uniform(self.WJSDWL[0], self.WJSDWL[1])

    gtz = [fxd, sd]

    # 使用启发式策略
    jtfs = random.uniform(0.5, 2.0)
    tfjg = random.uniform(1.2, 3.0)

    for i in range(3):
        tfs = jtfs + i * tfjg
        # 调整延时策略
        if i == 0:
            qby = random.uniform(2.0, 4.0)
        elif i == 1:
            qby = random.uniform(3.0, 6.0)
        else:
            qby = random.uniform(4.0, 8.0)

        gtz.extend([tfs, qby])

    return np.array(gtz)

def xgzt(self, gtz):
    """个体修复策略"""
    xg = gtz.copy()

    xg[1] = np.clip(xg[1], self.WJSDWL[0], self.WJSDWL[1])

    ywcs = xg[2:].reshape(3, 2)
    tfs = ywcs[:, 0]

```

```

tfs = np.maximum(tfs, 0)
pxcs = np.argsort(tfs)
pxts = tfs[pxcs]

tzts = [max(0.1, pxts[0])]

for i in range(1, len(pxts)):
    zxys = tzts[i - 1] + self.ZXTFJG
    if pxts[i] < zxys:
        tzts.append(zxys)
    else:
        tzts.append(pxts[i])

for i, y_idx in enumerate(pxcs):
    xg[2 + y_idx * 2] = tzts[i]

for i in range(3):
    ys_idx = 3 + i * 2
    xg[ys_idx] = np.clip(xg[ys_idx], 1.0, 10.0)

return xg

def zdsby(self, gtz, ds, zjds):
    """自适应变异"""
    by = gtz.copy()

    jd = ds / zjds
    byqd = 1.0 - 0.8 * jd

    if random.random() < self.BYLL:
        by[0] = (by[0] + random.gauss(0, 20 * byqd)) % 360

    if random.random() < self.BYLL:
        by[1] += random.gauss(0, 8 * byqd)

    for i in range(3):
        if random.random() < self.BYLL:
            by[2 + i * 2] += random.gauss(0, 0.8 * byqd)

        if random.random() < self.BYLL:
            by[3 + i * 2] += random.gauss(0, 0.4 * byqd)

    return self.xgzt(by)

def znjc(self, p1, p2):
    """智能交叉"""
    if random.random() > self.JCL:
        return p1.copy(), p2.copy()

    bl = random.uniform(0.3, 0.7)
    c1 = bl * p1 + (1 - bl) * p2
    c2 = (1 - bl) * p1 + bl * p2

    return self.xgzt(c1), self.xgzt(c2)

def jssx(self, zj, sxlfz, jss=5):

```

```

jssy = random.sample(range(len(zj)), jss)
jssxl = [sxlfx[i] for i in jssy]
yzidx = jssy[np.argmax(jssxl)]
return zj[yzidx].copy()

def yh(self):
    print("开始优化烟幕干扰弹投放策略...")
    print(f"算法参数: 种群={self.ZJDS}, 代数={self.DS}")
    print(f"约束: 投放间隔>={self.ZXTFJG}s, 速度{self.WJSDWL}m/s")
    print(f"目标模型: 半径{self.MBJJ}m、高{self.MBGD}m的圆柱体, {len(self.mbcy)}个采样点")
    print("=" * 70)

    self.kssj = time.time()

    # 初始化种群
    zj = []
    for _ in range(self.ZJDS):
        gtz = self.scznzt()
        zj.append(self.xgzt(gtz))

    zjsxl = -float('inf')
    zjgtz = None
    wgxs = 0

    for ds in range(self.DS):
        sxlfx = [self.jssxl(gtz) for gtz in zj]
        wcs = [self.jcys(gtz) for gtz in zj]
        yxgt = sum(1 for v in wcs if v == 0)

        pjsxl = np.mean(sxlfx)
        zdsxl = np.max(sxlfx)
        self.shlsj.append(pjsxl)
        self.zjshlsj.append(zdsxl)
        self.yswcs.append(len(wcs) - yxgt)

        zdsxlidx = np.argmax(sxlfx)
        dqzj = sxlfx[zdsxlidx]

        if dqzj > zjsxl:
            zjsxl = dqzj
            zjgtz = zj[zdsxlidx].copy()
            wgxs = 0

            csxl = self.jscsxl(zjgtz)
            yswc = wcs[zdsxlidx]

            print(f"第 {ds:3d} 代: 最佳={csxl:.3f}s, 平均={pjsxl:.3f}s, "
                  f"有效个体={yxgt}/{self.ZJDS}, 约束违反={yswc} ★ ")
        else:
            wgxs += 1
            if ds % 20 == 0:
                csxl = self.jscsxl(zjgtz) if zjgtz is not None else 0
                print(f"第 {ds:3d} 代: 最佳={csxl:.3f}s, 平均={pjsxl:.3f}s, "
                      f"有效个体={yxgt}/{self.ZJDS}")

```

```

# 早停检查
if wgxs >= self.ZTJC:
    print(f"\n 连续{self.ZTJC}代无改善，提前结束优化")
    break

# 生成新种群
xzj = []

# 精英保留
jyxd = [(i, sxlzf[i]) for i in range(len(zj)) if wcs[i] == 0]
jyxd.sort(key=lambda x: x[1], reverse=True)
jys = min(self.JYS, len(jyxd))

for i in range(jys):
    idx = jyxd[i][0]
    xzj.append(zj[idx].copy())

# 新个体
while len(xzj) < self.ZJDS:
    p1 = self.jssx(zj, sxlzf)
    p2 = self.jssx(zj, sxlzf)

    c1, c2 = self.znjc(p1, p2)

    xzj.append(self.zdsby(c1, ds, self.DS))
    if len(xzj) < self.ZJDS:
        xzj.append(self.zdsby(c2, ds, self.DS))

zj = xzj

yhsc = time.time() - self.kssj

if zjgtz is not None:
    zjcsxl = self.jscsxl(zjgtz)
    print(f"\n 优化完成! 最佳干扰时间: {zjcsxl:.3f}s")
    print(f"优化耗时: {yhsc:.2f} 秒")
else:
    print("\n 优化失败，未找到有效解")

print("=" * 70)
return zjgtz

def kshmbcy(self):
    """可视化"""
    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection='3d')

    zx_x, zx_y, zx_z = self.MBZX
    jj = self.MBJJ
    gd = self.MBGD

    theta = np.linspace(0, 2 * np.pi, 100)
    for z_offset in [0, gd / 2, gd]:
        z = zx_z - gd / 2 + z_offset
        x = zx_x + jj * np.cos(theta)
        y = zx_y + jj * np.sin(theta)

```

```

ax.plot(x, y, z, 'b-', alpha=0.3)

for j in [i * np.pi / 4 for i in range(8)]:
    x = [zx_x + jj * np.cos(j), zx_x + jj * np.cos(j)]
    y = [zx_y + jj * np.sin(j), zx_y + jj * np.sin(j)]
    z = [zx_z - gd / 2, zx_z + gd / 2]
    ax.plot(x, y, z, 'b-', alpha=0.3)

cy_x = [p[0] for p in self.mbcy]
cy_y = [p[1] for p in self.mbcy]
cy_z = [p[2] for p in self.mbcy]
ax.scatter(cy_x, cy_y, cy_z, c='r', marker='o', s=50, label='采样点')

ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Z (m)')
ax.set_title(f'圆柱形目标及采样点 (半径={self.MBJJ}m, 高={self.MBGD}m)')
ax.legend(loc='upper left', bbox_to_anchor=(1.02, 1), borderaxespad=0) # 图例放在图外

plt.tight_layout()
plt.savefig('target_samples.png', dpi=300, bbox_inches='tight')
plt.close()
print(f'圆柱形目标及{len(self.mbcy)}个优化采样点已保存为 target_samples.png')

def ksyhgc(self):
    if not self.shlsj:
        print("没有优化历史数据可显示")
        return

fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(15, 10))

# 1. 适应度演化
ds = range(len(self.shlsj))
ax1.plot(ds, self.shlsj, 'b-', label='平均适应度', alpha=0.7)
ax1.plot(ds, self.zjshlsj, 'r-', label='最佳适应度', linewidth=2)
ax1.set_xlabel('代数')
ax1.set_ylabel('适应度 (秒)')
ax1.set_title('适应度演化过程')
ax1.legend(loc='upper right')
ax1.grid(True, alpha=0.3)

# 2. 约束违反
ax2.bar(ds, self.yswcs, alpha=0.6, color='orange')
ax2.set_xlabel('代数')
ax2.set_ylabel('违反约束的个体数')
ax2.set_title('约束违反统计')
ax2.grid(True, alpha=0.3)

# 3. 收敛性
if len(self.zjshlsj) > 10:
    window = 10
    hpc = []
    for i in range(window, len(self.zjshlsj)):
        hpc.append(np.mean(self.zjshlsj[i - window:i]))

```

```

ax3.plot(range(window, len(self.zjshlsj)), hpc, 'g-', linewidth=2)
ax3.set_xlabel('代数')
ax3.set_ylabel('滑动平均适应度')
ax3.set_title(f'收敛性分析（{window}代滑动平均）')
ax3.grid(True, alpha=0.3)

# 4. 适应度改进率
if len(self.zjshlsj) > 1:
    gjl = []
    for i in range(1, len(self.zjshlsj)):
        if self.zjshlsj[i - 1] != 0:
            l = (self.zjshlsj[i] - self.zjshlsj[i - 1]) / \
                self.zjshlsj[i - 1] * 100
            gjl.append(l)
        else:
            gjl.append(0)

    ax4.plot(range(1, len(self.zjshlsj)), gjl, 'purple', alpha=0.7)
    ax4.axhline(y=0, color='black', linestyle='--', alpha=0.5)
    ax4.set_xlabel('代数')
    ax4.set_ylabel('改进率 (%)')
    ax4.set_title('适应度改进率')
    ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('optimization_progress.png', dpi=300, bbox_inches='tight')
plt.close() # 只保存不显示，避免重复
print("优化过程可视化已保存为 optimization_progress.png")

def ksscene3d(self, zjgtz, sjfw=None):
    if zjgtz is None:
        print("没有最优解可显示")
        return

    fxd, sd = zjgtz[:2]
    ywcs = zjgtz[2:].reshape(3, 2)
    wjsd = self.qdwjsdxl(fxd, sd)

    if sjfw is None:
        sjfw = (0, min(30, self.ZDSJ))

    fig = plt.figure(figsize=(16, 12))
    ax = fig.add_subplot(111, projection='3d')

    ax.scatter(*self.FY1CS, c='blue', s=100, marker='^', label='FY1 无人机初始位置')

    ax.scatter(*self.M1CS, c='red', s=100, marker='s', label='M1 导弹初始位置')

    ax.scatter(*self.JMB, c='black', s=150, marker='*', label='假目标')

    zx_x, zx_y, zx_z = self.MBZX
    jj = self.MBJJ
    gd = self.MBGD

    theta = np.linspace(0, 2 * np.pi, 50)

```

```

for z_val in [zx_z - gd / 2, zx_z + gd / 2]:
    x_circle = zx_x + jj * np.cos(theta)
    y_circle = zx_y + jj * np.sin(theta)
    z_circle = np.full_like(x_circle, z_val)
    ax.plot(x_circle, y_circle, z_circle, 'g-', alpha=0.6, linewidth=2)

for j in np.linspace(0, 2 * np.pi, 8, endpoint=False):
    x_line = [zx_x + jj * np.cos(j)] * 2
    y_line = [zx_y + jj * np.sin(j)] * 2
    z_line = [zx_z - gd / 2, zx_z + gd / 2]
    ax.plot(x_line, y_line, z_line, 'g-', alpha=0.6, linewidth=1)

sj = np.linspace(sjfw[0], sjfw[1], 200)

# 导弹轨迹
dldwz = np.array([self.qdldwz(t) for t in sj])
ax.plot(dldwz[:, 0], dldwz[:, 1], dldwz[:, 2],
        'r-', linewidth=3, alpha=0.8, label='M1 导弹轨迹')

# 无人机轨迹
wjwz = np.array([self.qdwjwz(t, fxd, sd) for t in sj])
ax.plot(wjwz[:, 0], wjwz[:, 1], wjwz[:, 2],
        'b--', linewidth=2, alpha=0.8, label='FY1 无人机轨迹')

# 绘制烟幕弹轨迹和遮蔽区域
ys = ['orange', 'purple', 'brown']
for i, (tfs, qby) in enumerate(ywcs):
    y = ys[i]

    tfwz = self.qdwjwz(tfs, fxd, sd)
    ax.scatter(*tfwz, c=y, s=80, marker='o',
              label=f'烟幕弹 {i + 1} 投放点 t={tfs:.1f}s')

    qbsj = tfs + qby
    qbwz = qywzdk(tfwz, wjsd, qby, 0,
                  self.ZL, self.YWXSD)
    ax.scatter(*qb wz, c=y, s=120, marker='*',
              label=f'烟幕弹 {i + 1} 起爆点 t={qbsj:.1f}s')

    xlsj = np.linspace(0, qby, 50)
    xlwz = []
    for dt in xlsj:
        wz = qywzdk(tfwz, wjsd, dt, dt - qby,
                    self.ZL, self.YWXSD)
        if not np.any(np.isnan(wz)):
            xlwz.append(wz)

    if xlwz:
        xlwz = np.array(xlwz)
        ax.plot(xlwz[:, 0], xlwz[:, 1], xlwz[:, 2],
                color=y, linestyle=':', linewidth=2, alpha=0.7)

    xcsj = np.linspace(0, self.YXSJ, 50)
    xcwz = []
    for dt in xcsj:
        wz = qywzdk(tfwz, wjsd, qby, dt,

```

```

        self.ZL, self.YWXSD)
    if not np.any(np.isnan(wz)):
        xcwz.append(wz)

    if xcwz:
        xcwz = np.array(xcwz)
        ax.plot(xcwz[:, 0], xcwz[:, 1], xcwz[:, 2],
                color=y, linestyle='-', linewidth=3, alpha=0.8)

# 坐标轴
ax.set_xlabel('X 坐标 (m)')
ax.set_ylabel('Y 坐标 (m)')
ax.set_zlabel('Z 坐标 (m)')
ax.set_title('烟幕干扰弹投放策略 3D 场景可视化')

ax.view_init(elev=20, azim=45)

ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.savefig('3d_scene.png', dpi=300, bbox_inches='tight')
plt.close()
print("3D 场景可视化已保存为 3d_scene.png")

def ksbgjsj(self, zjgtz, xqsjfw=None):
    """可视化遮蔽时间线"""
    if zjgtz is None:
        print("没有最优解可显示")
        return

    fxd, sd = zjgtz[:2]
    ywcs = zjgtz[2:].reshape(3, 2)
    wjsd = self.qdwjsdxi(fxd, sd)

    if xqsjfw is None:
        xqsjfw = (0, min(40, self.ZDSJ))

    # 计算详细的遮蔽时间线
    sjjc = 0.02
    sj = np.arange(xqsjfw[0], xqsjfw[1], sjjc)

    zjbg = []
    dgybg = [[] for _ in range(3)]
    dljl = []

    for t in sj:
        dldwz = self.qdldwz(t)
        zjmb = np.linalg.norm(dldwz - self.MBZX)
        dljl.append(zjmb)

    yzjbg = False

    for ywid, (tfs, qby) in enumerate(ywcs):
        t_fs = t - tfs
        t_qb = t - tfs - qby

```



```

ydgbg = False

if (t_fs >= 0 and t_qb >= 0 and t_qb <= self.YXSJ):
    wjwz = self.qdwjwz(tfs, fxd, sd)
    ywz = qywzdk(
        wjwz, wjsd, t_fs, t_qb,
        self.ZL, self.YWXSD
    )

    if jccyggzk(
        dldwz, ywz, self.mbcy, self.YXJZ
    ):
        ydgbg = True
        yzjbg = True

dgybg[ywid].append(1 if ydgbg else 0)

zjbg.append(1 if yzjbg else 0)

# 图表
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(15, 12))

# 1. 总体遮蔽时间线
ax1.fill_between(sj, 0, zjbg, alpha=0.7, color='red', label='总体遮蔽')
ax1.set_ylabel('遮蔽状态')
ax1.set_title('总体遮蔽时间线')
ax1.grid(True, alpha=0.3)
ax1.set_ylim(-0.1, 1.1)

for i, (tfs, qby) in enumerate(ywcs):
    qbsj = tfs + qby
    jssj = qbsj + self.YXSJ

    ax1.axvline(tfs, color=f'C {i}', linestyle='--', alpha=0.7,
        label=f'烟幕弹 {i + 1} 投放')
    ax1.axvline(qbsj, color=f'C {i}', linestyle=':', alpha=0.7,
        label=f'烟幕弹 {i + 1} 起爆')

ax1.legend(loc='upper right')

# 2. 各烟幕弹的单独遮蔽效果
ys = ['orange', 'purple', 'brown']
for i in range(3):
    ax2.fill_between(sj, i, np.array(dgybg[i]) + i,
        alpha=0.6, color=ys[i], label=f'烟幕弹 {i + 1}')

ax2.set_ylabel('烟幕弹编号')
ax2.set_title('各烟幕弹单独遮蔽效果')
ax2.set_yticks(range(3))
ax2.set_yticklabels(['烟幕弹 1', '烟幕弹 2', '烟幕弹 3'])
ax2.grid(True, alpha=0.3)
ax2.legend(loc='upper right') # 图例放在右上角

# 3. 导弹到目标距离
ax3.plot(sj, dljl, 'b-', linewidth=2, label='导弹到目标距离')

```

```

ax3.axhline(y=50, color='red', linestyle='--', alpha=0.7, label='警戒距离(50m)')
ax3.set_xlabel('时间 (秒)')
ax3.set_ylabel('距离 (米)')
ax3.set_title('导弹到目标距离变化')
ax3.grid(True, alpha=0.3)
ax3.legend(loc='upper right') # 图例放在右上角

plt.tight_layout()
plt.savefig('coverage_time.png', dpi=300, bbox_inches='tight')
plt.close() # 只保存不显示，避免重复
print("遮蔽时间线可视化已保存为 coverage_time.png")

zjyx = sum(zjbg) * sjjc
dgyx = [sum(bg) * sjjc for bg in dgybg]

print(f"\n 遮蔽效果统计:")
print(f"总有效遮蔽时间: {zjyx:.3f} 秒")
for i, t in enumerate(dgyx):
    print(f"烟幕弹 {i + 1} 单独遮蔽时间: {t:.3f} 秒")

def ksywgi(self, zjgtz):
    """可视化烟幕弹详细轨迹"""
    if zjgtz is None:
        print("没有最优解可显示")
        return

    fxd, sd = zjgtz[:2]
    ywcs = zjgtz[2:].reshape(3, 2)
    wjsd = self.qdwjsdxi(fxd, sd)

    fig = plt.figure(figsize=(18, 6))

    ys = ['orange', 'purple', 'brown']

    for i, (tfs, qby) in enumerate(ywcs):
        ax = fig.add_subplot(1, 3, i + 1, projection='3d')

        tfwz = self.qdwjwz(tfs, fxd, sd)
        ax.scatter(*tfwz, c=ys[i], s=100, marker='o', label='投放点')

        xlsj = np.linspace(0, qby, 100)
        xlwz = []
        for dt in xlsj:
            wz = qywzdk(tfwz, wjsd, dt, dt - qby,
                        self.ZL, self.YWXSD)
            if not np.any(np.isnan(wz)):
                xlwz.append(wz)

        if xlwz:
            xlwz = np.array(xlwz)
            ax.plot(xlwz[:, 0], xlwz[:, 1], xlwz[:, 2],
                    'b--', linewidth=2, alpha=0.7, label='下落轨迹')

        qbwz = qywzdk(tfwz, wjsd, qby, 0,
                      self.ZL, self.YWXSD)

```

```

ax.scatter(*qbwz, c='red', s=150, marker='*', label='起爆点')

xcsj = np.linspace(0, self.YXSJ, 100)
xcwz = []
for dt in xcsj:
    wz = qywzdk(tfwz, wjsd, qby, dt,
                 self.ZL, self.YWXSD)
    if not np.any(np.isnan(wz)):
        xcwz.append(wz)

if xcwz:
    xcwz = np.array(xcwz)
    ax.plot(xcwz[:, 0], xcwz[:, 1], xcwz[:, 2],
            ys[i], linewidth=3, alpha=0.8, label='下沉轨迹')

# 在几个关键时间点绘制有效范围球体
gjsj = [0, self.YXSJ / 4, self.YXSJ / 2,
        3 * self.YXSJ / 4, self.YXSJ]
for t_idx, dt in enumerate(gjsj):
    if dt <= self.YXSJ:
        wz = qywzdk(tfwz, wjsd, qby, dt,
                     self.ZL, self.YWXSD)
        if not np.any(np.isnan(wz)):
            # 绘制有效范围球体（简化为几个圆圈）
            u = np.linspace(0, 2 * np.pi, 20)
            for phi in [0, np.pi / 2]:
                x = wz[0] + self.YXJZ * np.cos(u) * np.cos(phi)
                y = wz[1] + self.YXJZ * np.sin(u) * np.cos(phi)
                z = wz[2] + self.YXJZ * np.sin(phi) * np.ones_like(u)
                ax.plot(x, y, z, ys[i], alpha=0.2, linewidth=1)

# 绘制目标位置
ax.scatter(*self.MBZX, c='green', s=50, marker='s', alpha=0.5, label='目标中心')

ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Z (m)')
ax.set_title(f'烟雾弹 {i + 1} 详细轨迹\n 投放 {tfs:.1f}s, 延时 {qby:.1f}s')
ax.legend(loc='upper left', bbox_to_anchor=(1.02, 1), borderaxespad=0) # 图例放在图外
ax.view_init(elev=15, azim=45)

plt.tight_layout()
plt.savefig('smoke_trajectory.png', dpi=300, bbox_inches='tight')
plt.close() # 只保存不显示，避免重复
print("烟雾弹轨迹详情已保存为 smoke_trajectory.png")

def sczhbg(self, zjgtz):
    if zjgtz is None:
        print("没有最优解，无法生成报告")
        return

    fxd, sd = zjgtz[:2]
    ywcs = zjgtz[2:].reshape(3, 2)
    dqsj, dgyx = self.jsbjgjdk(zjgtz)

# 报告图表

```

```

fig = plt.figure(figsize=(20, 16))

# 1. 策略
ax1 = plt.subplot(3, 3, 1)
cl = [
    ['飞行方向', f'{fxd:.1f}°'],
    ['飞行速度', f'{sd:.1f} m/s'],
    ['总遮蔽时间', f'{dqsj:.3f}s'],
    ['烟幕弹数量', '3 枚']
]

table = ax1.table(cellText=cl,
                  colLabels=['参数', '数值'],
                  cellLoc='center', loc='center',
                  colWidths=[0.5, 0.5])
table.auto_set_font_size(False)
table.set_fontsize(12)
table.scale(1, 2)
ax1.axis('off')
ax1.set_title('策略概览', fontsize=14, fontweight='bold')

# 2. 烟幕弹详情
ax2 = plt.subplot(3, 3, 2)
ywsj = []
for i, (tfs, qby) in enumerate(ywcs):
    ywsj.append([
        f'烟幕弹{i + 1}',
        f'{tfs:.2f}s',
        f'{qby:.2f}s',
        f'{dgyx[i]:.3f}s'
    ])

table2 = ax2.table(cellText=ywsj,
                   colLabels=['编号', '投放时间', '起爆延时', '遮蔽时间'],
                   cellLoc='center', loc='center',
                   colWidths=[0.2, 0.25, 0.25, 0.3])
table2.auto_set_font_size(False)
table2.set_fontsize(10)
table2.scale(1, 1.5)
ax2.axis('off')
ax2.set_title('烟幕弹详情', fontsize=14, fontweight='bold')

# 3. 优化收敛
ax3 = plt.subplot(3, 3, 3)
if self.zjshlsj:
    ax3.plot(self.zjshlsj, 'r-', linewidth=2, label='最佳适应度')
    ax3.set_xlabel('迭代代数')
    ax3.set_ylabel('最佳适应度 (秒)')
    ax3.set_title('优化收敛过程')
    ax3.grid(True, alpha=0.3)
    ax3.legend(loc='upper right') # 图例放在右上角

# 4. 3D 场景
ax4 = plt.subplot(3, 3, 4, projection='3d')
ax4.scatter(*self.FY1CS, c='blue', s=50, marker='^', label='FY1')

```

```

ax4.scatter(*self.M1CS, c='red', s=50, marker='s', label='M1')
ax4.scatter(*self.MBZX, c='green', s=50, marker='o', label='目标')

# 无人机轨迹
ts = np.linspace(0, 10, 50)
wjgj = np.array([self.qdwjwz(t, fxd, sd) for t in ts])
ax4.plot(wjgj[:, 0], wjgj[:, 1], wjgj[:, 2], 'b--', alpha=0.6)

# 导弹轨迹
dlgj = np.array([self.qdldwz(t) for t in ts])
ax4.plot(dlgj[:, 0], dlgj[:, 1], dlgj[:, 2], 'r-', alpha=0.6)

ax4.set_title('场景概览')
ax4.legend(loc='upper left', bbox_to_anchor=(1.02, 1), borderaxespad=0) # 图例放在图外

# 5. 时间效率
ax5 = plt.subplot(3, 3, 5)
bq = ['烟幕弹 1', '烟幕弹 2', '烟幕弹 3']
sz = dgyx
ys = ['orange', 'purple', 'brown']

bars = ax5.bar(bq, sz, color=ys, alpha=0.7)
ax5.set_ylabel('遮蔽时间 (秒)')
ax5.set_title('各烟幕弹遮蔽效果')
ax5.grid(True, alpha=0.3, axis='y')

for bar, sz in zip(bars, sz):
    height = bar.get_height()
    ax5.text(bar.get_x() + bar.get_width() / 2., height + 0.01,
             f'{sz:.3f}s', ha='center', va='bottom')

# 6. 约束
ax6 = plt.subplot(3, 3, 6)
ys = [
    f'速度: {sd:.1f} m/s\n 范围: {self.WJSDWL[0]}-{self.WJSDWL[1]} m/s',
    f'投放间隔 1-2: {ywcs[1, 0] - ywcs[0, 0]:.2f}s\n 最小要求: {self.ZXTFJG}s',
    f'投放间隔 2-3: {ywcs[2, 0] - ywcs[1, 0]:.2f}s\n 最小要求: {self.ZXTFJG}s',
    f'起爆延时: {ywcs[:, 1].min():.1f}-{ywcs[:, 1].max():.1f}s\n 要求范围: 1.0-10.0s'
]

zt = ['✓', '✓', '✓', '✓'] # 假设都满足约束

for i, (ys, zt) in enumerate(zip(ys, zt)):
    color = 'green' if zt == '✓' else 'red'
    ax6.text(0.05, 0.8 - i * 0.2, f'{zt} {ys}',
             transform=ax6.transAxes, fontsize=10, color=color,
             verticalalignment='top')

ax6.set_xlim(0, 1)
ax6.set_ylim(0, 1)
ax6.axis('off')
ax6.set_title('约束满足情况', fontsize=14, fontweight='bold')

# 7. 投放时序图
ax7 = plt.subplot(3, 3, 7)

```

```

tfs = ywcs[:, 0]
qbsj = tfs + ywcs[:, 1]
jssj = qbsj + self.YXSJ

for i in range(3):
    ax7.barh(i, ywcs[i, 1], left=tfs[i],
             color=ys[i], alpha=0.3, label=f'烟幕弹 {i + 1}准备期')
    ax7.barh(i, self.YXSJ, left=qbsj[i],
             color=ys[i], alpha=0.8, label=f'烟幕弹 {i + 1}有效期')

ax7.set_xlabel('时间 (秒)')
ax7.set_ylabel('烟幕弹编号')
ax7.set_yticks(range(3))
ax7.set_yticklabels(['烟幕弹 1', '烟幕弹 2', '烟幕弹 3'])
ax7.set_title('投放时序图')
ax7.grid(True, alpha=0.3, axis='x')
ax7.legend(loc='upper right') # 图例放在右上角

# 8. 空间覆盖效率
ax8 = plt.subplot(3, 3, 8)
zjtk = 3 * (4 / 3) * np.pi * (self.YXJZ ** 3) # 三个球体总体积
yxtkbl = dqsj / (sum(dgyx) + 1e-6) # 避免除零

zbs = ['时间效率', '空间利用率', '协同效果']
gysz = [
    dqsj / 10, # 归一化到 0-1
    yxtkbl,
    dqsj / sum(dgyx) if sum(dgyx) > 0 else 0
]

jd = np.linspace(0, 2 * np.pi, len(zbs), endpoint=False).tolist()
gysz += gysz[1:] # 闭合
jd += jd[1:]

ax8.plot(jd, gysz, 'o-', linewidth=2, color='blue', label='性能指标')
ax8.fill(jd, gysz, alpha=0.25, color='blue')
ax8.set_xticks(jd[:-1])
ax8.set_xticklabels(zbs)
ax8.set_ylim(0, 1)
ax8.set_title('性能雷达图')
ax8.grid(True)
ax8.legend(loc='upper right') # 图例放在右上角

# 9. 总结评价
ax9 = plt.subplot(3, 3, 9)

# 评价
xlsx = (dqsj / 10) * 100 # 假设 10 秒为满分
pj = f''''

ax9.text(0.05, 0.95, pj, transform=ax9.transAxes,
        fontsize=10, verticalalignment='top', fontfamily='monospace')
ax9.set_xlim(0, 1)
ax9.set_ylim(0, 1)
ax9.axis('off')

```

```

ax9.set_title('综合评价', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.savefig('report.png', dpi=300, bbox_inches='tight')
plt.close() # 只保存不显示，避免重复
print("综合报告已保存为 report.png")

def dczjdxls(self, zjgtz, wjm='results.xlsx'):
    if zjgtz is None:
        print("没有最优解可导出")
        return

    fxd, sd = zjgtz[:2]
    ywcs = zjgtz[2:].reshape(3, 2)
    wjsd = self.qdwjsdxl(fxd, sd)

    zjsj = []
    for i, (tfs, qby) in enumerate(ywcs):
        tfwz = self.qdwjwz(tfs, fxd, sd)

        qbwz = qywzdk(tfwz, wjsd, qby, 0,
                      self.ZL, self.YWXSD)

        zjsj.append({
            '烟幕弹编号': i + 1,
            '无人机编号': 'FY1',
            '飞行方向(度)': fxd,
            '飞行速度(m/s)': sd,
            '投放时间(s)': tfs,
            '投放点 X(m)': tfwz[0],
            '投放点 Y(m)': tfwz[1],
            '投放点 Z(m)': tfwz[2],
            '起爆延时(s)': qby,
            '起爆点 X(m)': qbwz[0],
            '起爆点 Y(m)': qbwz[1],
            '起爆点 Z(m)': qbwz[2]
        })

    # 创建 DataFrame 保存
    df = pd.DataFrame(zjsj)

    flcx = ['飞行方向(度)', '飞行速度(m/s)', '投放时间(s)', '投放点 X(m)',
            '投放点 Y(m)', '投放点 Z(m)', '起爆延时(s)', '起爆点 X(m)',
            '起爆点 Y(m)', '起爆点 Z(m)']

    for col in flcx:
        if col in ['投放时间(s)', '起爆延时(s)']:
            df[col] = df[col].round(2)
        else:
            df[col] = df[col].round(1)

    df.to_excel(wjm, index=False)
    print(f"结果已导出到 {wjm}")

```

```

# 显示数据
print("\n 导出数据预览:")
print(df.to_string(index=False))

def gsjg(self, zjgtz):
    if zjgtz is None:
        print("无有效解可显示")
        return None

    fxd, sd = zjgtz[:2]
    ywcs = zjgtz[2:].reshape(3, 2)
    wjsd = self.qdwjsdxl(fxd, sd)

    dqsj, dgyx = self.jsbjgjkd(zjgtz)

    print("\n=== 最优策略详情 ===")
    print(f"无人机 FY1 飞行方向: {fxd:.1f} ° ")
    print(f"无人机 FY1 飞行速度: {sd:.1f} m/s")
    print(f"总有效遮蔽时间（去重后）: {dqsj:.3f}s")
    print()

    jg = []
    for i, (tfs, qby) in enumerate(ywcs):
        wjwz = self.qdwjwz(tfs, fxd, sd)

        qbsj = qby
        x_qb = wjwz[0] + wjsd[0] * qbsj
        y_qb = wjwz[1] + wjsd[1] * qbsj
        z_qb = wjwz[2] - 0.5 * self.ZL * qbsj**2
        qbwz = np.array([x_qb, y_qb, z_qb])

        jg.append({
            'ywid': i + 1,
            'tfs': tfs,
            'qby': qby,
            'tfwz': wjwz,
            'qbwz': qbwz,
            'dgyx': dgyx[i]
        })

    print(f"烟幕弹 {i + 1}:")
    print(f" 投放时间: {tfs:.2f}s")
    print(f" 起爆延时: {qby:.2f}s")
    print(f" 投放点: ({wjwz[0]:.1f}, {wjwz[1]:.1f}, {wjwz[2]:.1f})")
    print(f" 起爆点: ({qbzw[0]:.1f}, {qbzw[1]:.1f}, {qbzw[2]:.1f})")
    print(f" 单独遮蔽时间: {dgyx[i]:.3f}s")
    print()

    print(f"总有效遮蔽时间（去重后）: {dqsj:.3f}s")

    return {
        'fxd': fxd,
        'sd': sd,
        'jg': jg,
        'zjyx': dqsj,
    }

```



```

        'dgyx': dgyx
    }

def yzjg(self, zjgtz):
    if zjgtz is None:
        print("\n=== 无解可验证 ===")
        return False

    fxd, sd = zjgtz[:2]
    ywcs = zjgtz[2:].reshape(3, 2)

    print("\n=== 解的可行性验证 ===")

    qyzx = True

    sdyd = self.WJSDWL[0] <= sd <= self.WJSDWL[1]
    print(
        f"飞行速度: {sd:.1f} m/s (范围: {self.WJSDWL[0]}-{self.WJSDWL[1]}) {'✓' if sdyd else '✗'}")
    qyzx &= sdyd

    tfs = ywcs[:, 0]
    for i in range(1, 3):
        jg = tfs[i] - tfs[i - 1]
        jgyd = jg >= self.ZXTFJG
        print(f"烟幕弹 {i} 与 {i + 1} 投放间隔: {jg:.2f}s {'✓' if jgyd else '✗'}")
        qyzx &= jgyd

    for i, (_, qby) in enumerate(ywcs):
        ydyd = 1.0 <= qby <= 10.0
        print(f"烟幕弹 {i + 1} 起爆延时: {qby:.2f}s {'✓' if ydyd else '✗'}")
        qyzx &= ydyd

    print(f"\n 整体可行性: {'通过' if qyzx else '不通过'}")
    return qyzx

def scsyksh(self, zjgtz):
    if zjgtz is None:
        print("没有最优解，无法生成可视化")
        return

    print("\n=== 开始生成可视化图表 ===")

    print("1. 生成目标采样点可视化...")
    self.kshmbcy()

    print("2. 生成优化过程可视化...")
    self.ksyhgc()

    print("3. 生成 3D 场景可视化...")
    self.ksscene3d(zjgtz)

    print("4. 生成遮蔽时间线可视化...")
    self.ksbgsj(zjgtz)

```

```

print("5. 生成烟幕弹轨迹详情...")
self.ksywgj(zjgtz)

print("6. 生成综合报告...")
self.sczhbg(zjgtz)

print("\n=== 所有可视化图表生成完成 ===")
print("生成的文件:")
print("- target_samples.png")
print("- optimization_progress.png")
print("- 3d_scene.png")
print("- coverage_time.png")
print("- smoke_trajectory.png")
print("- report.png")

def zhu():
    print("烟幕干扰弹投放策略优化系统 - 完整可视化版")
    print("=" * 70)
    print("问题: 利用无人机 FY1 投放 3 枚烟幕干扰弹实施对 M1 的干扰")
    print("目标: 最大化有效遮蔽时间（去除重叠时间）")
    print("算法: 增强遗传算法 + 多重优化策略 + 完整可视化")
    print("优化: 快速计算、智能交叉、自适应变异、分层扫描")
    print("=" * 70)

    # 随机种子
    random.seed(42)
    np.random.seed(42)

    yhq = YHGYDCYHZSJ()

    yhq.kshmbcy()

    zjjg = yhq.yh()

    if zjjg is not None:
        sfyx = yhq.yzjg(zjjg)

        if sfyx:
            df = yhq.jscsxl(zjjg)
            print(f"找到有效解, 得分: {df:.3f}s")

            jgsj = yhq.gsjg(zjjg)

            yhq.scsyksh(zjjg)

            yhq.dczjdxls(zjjg, 'results.xlsx')

            print("\n=== 优化效果分析 ===")
            fxd, sd = zjjg[:2]
            ywcs = zjjg[2:].reshape(3, 2)

            print(f"飞行策略: 方向{direction:.1f}°, 速度{speed:.1f}m/s")
            print("投放序列:")
            for i, (tfs, qby) in enumerate(ywcs):

```

```

        print(f' 烟幕弹 {i + 1}: t={tfs:.2f}s 投放, 延时 {qby:.2f}s 起爆')

    tfs = ywcs[:, 0]
    jg = [tfs[i] - tfs[i - 1] for i in range(1, len(tfs))]
    print(f'投放间隔: {jg[0]:.2f}s, {jg[1]:.2f}s')

    print("\n  优化完成! 所有结果已保存到文件和图表中。")

    else:
        print("解不满足约束条件")
    else:
        print("未找到有效解")

if __name__ == "__main__":
    zhu()

```

代码4： 问题四求解

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import random
import time
import warnings

warnings.filterwarnings('ignore')

# 中文字体
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

def juli3d(p1, p2):
    return np.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2 + (p1[2] - p2[2])**2)

def xqjqc(p1, p2, qz, bj):
    """线段-球体相交检测"""
    xc = p2 - p1
    p1_to_qz = qz - p1

    xc_cf = np.dot(xc, xc)
    if xc_cf == 0:
        return juli3d(p1, qz) <= bj

    t = np.dot(p1_to_qz, xc) / xc_cf

    if t < 0.0:
        jjd = p1
    elif t > 1.0:
        jjd = p2
    else:
        jjd = p1 + t * xc

```

```

jl = juli3d(jjd, qz)
return jl <= bj

def ywz(pfwz, pfsd, t_tc, t_qb, zl, ywcjsd):
    """计算烟幕位置"""
    if t_qb < 0:
        x = pfwz[0] + pfsd[0] * t_tc
        y = pfwz[1] + pfsd[1] * t_tc
        z = pfwz[2] - 0.5 * zl * t_tc * t_tc
        return np.array([x, y, z])
    else:
        qbsj = t_tc - t_qb
        x_qb = pfwz[0] + pfsd[0] * qbsj
        y_qb = pfwz[1] + pfsd[1] * qbsj
        z_qb = pfwz[2] - 0.5 * zl * qbsj * qbsj

        x = x_qb
        y = y_qb
        z = z_qb - ywcjsd * t_qb

        return np.array([x, y, z])

def cybz(ddwz, ywz, bz, yxbj):
    """采样点是否被烟幕遮蔽"""
    if np.any(np.isnan(ywz)):
        return False

    for y in bz:
        if not xqjqjc(ddwz, y, ywz, yxbj):
            return False
    return True

class XZDWRJYH:

    def __init__(self):

        self.DDSD = 300
        self.YWCJSD = 3
        self.YXBJ = 10
        self.YXSY = 20
        self.BZBJ = 7
        self.BZG = 10
        self.WRSD = [70, 140]
        self.ZL = 9.8

        # 初始位置
        self.WRCswz = {
            "FY1": np.array([17800, 0, 1800]),
            "FY2": np.array([12000, 1400, 1400]),
            "FY3": np.array([6000, -3000, 700])
        }
        self.DDCswz = np.array([20000, 0, 2000])
        self.JMBD = np.array([0, 0, 0])
        self.ZMZX = np.array([0, 200, self.BZG / 2])

```

```

self.bz = self.scyzb()

# 算法参数
self.CLCS = 150
self.YCS = 200
self.BYL = 0.1
self.JCL = 0.8
self.JYS = 30
self.TZCS = 50

# 时间参数
self.SJJG = 0.1
self.ZDSJ = 70

self.yjddj()

self.sydls = []
self.zjsydls = []

def scyzb(self):
    """生成圆柱形目标采样点"""
    samples = []
    zx_x, zx_y, zx_z = self.ZMZX
    bj = self.BZBJ
    gd = self.BZG

    samples.append(np.array([zx_x, zx_y, zx_z]))

    jd = [0, np.pi / 3, 2 * np.pi / 3, np.pi, 4 * np.pi / 3, 5 * np.pi / 3]

    for z_offs in [0, gd]:
        z = zx_z - gd / 2 + z_offs
        for j in jd:
            x = zx_x + bj * np.cos(j)
            y = zx_y + bj * np.sin(j)
            samples.append(np.array([x, y, z]))

    z = zx_z
    for j in jd:
        x = zx_x + bj * np.cos(j)
        y = zx_y + bj * np.sin(j)
        samples.append(np.array([x, y, z]))

    return np.array(samples)

def yjddj(self):
    """预计算导弹轨迹"""
    fx = self.JMBD - self.DDCSWZ
    jl = np.linalg.norm(fx)
    self.ddfx = fx / jl if jl > 0 else np.array([0, 0, 0])
    self.ddzdsj = jl / self.DDSD
    print(f"导弹总飞行时间: {self.ddzdsj:.1f}秒")

def hdwzdw(self, t):

```

```

return self.DDCSWZ + self.ddfx * self.DDSD * t

def hdwrwz(self, wrid, t, fxj, sd):
    fxj_rad = np.radians(fxj)
    fx = np.array([np.cos(fxj_rad), np.sin(fxj_rad), 0])
    return self.WRCSWZ[wrid] + fx * sd * t

def hdwrsd(self, fxj, sd):
    fxj_rad = np.radians(fxj)
    return np.array([
        sd * np.cos(fxj_rad),
        sd * np.sin(fxj_rad),
        0
    ])

def sccigt(self):
    gt = []

    jcsj = [15, 30, 45]

    for i, (wrid, mbsj) in enumerate(zip(["FY1", "FY2", "FY3"], jcsj)):
        if mbsj < self.ddzdsj:
            ddwz = self.hdwzdw(mbsj)
            wrwz = self.WRCSWZ[wrid]

            fxnl = ddwz - wrwz
            jdfx = np.degrees(np.arctan2(fxnl[1], fxnl[0]))
            if jdfx < 0:
                jdfx += 360

            cs_tc = random.uniform(2, 8)
            cs_sd = random.uniform(70, 140)
            cs_wz = self.hdwrwz(wrid, cs_tc, jdfx, cs_sd)

            if cs_wz[1] < 0:
                if wrwz[1] < 0:
                    fx = random.uniform(30, 150)
                else:
                    fx = random.uniform(-30, 30)
            else:
                fx = (jdfx + random.uniform(-30, 30)) % 360

            # 计算合适的速度和时间
            hxjl = np.linalg.norm(fxnl[:2])
            qbyc = random.uniform(4, 8)
            tcsj = max(1, mbsj - qbyc - hxjl / 100)

            xysd = hxjl / max(tcsj, 1)
            sd = np.clip(xysd + random.uniform(-20, 20), 70, 140)
        else:
            wrwz = self.WRCSWZ[wrid]
            if wrwz[1] < 0:
                fx = random.uniform(30, 150)
            else:
                fx = random.uniform(0, 180)
            sd = random.uniform(70, 140)
            tcsj = random.uniform(1, 20)

```

```

        qbyc = random.uniform(3, 10)

        gt.extend([fx, sd, tcsj, qbyc])

    return np.array(gt)

def jszcbsj(self, gt):
    """计算遮蔽时间"""
    cs = gt.reshape(3, 4)
    zzc = set()

    for t in np.arange(0, self.ZDSJ, self.SJJG):
        ddwz = self.hdwzdw(t)

        for i, wrid in enumerate(["FY1", "FY2", "FY3"]):
            fx, sd, tcsj, qbyc = cs[i]

            t_tc = t - tcsj
            t_qb = t - tcsj - qbyc

            # 烟幕有效条件
            if (t_tc >= 0 and t_qb >= 0 and
                t_qb <= self.YXSY):

                wrwz = self.hdwrrwz(wrid, tcsj, fx, sd)
                sdxl = self.hdwrsd(fx, sd)
                ywz_pos = ywz(
                    wrwz, sdxl, t_tc, t_qb,
                    self.ZL, self.YWCJSD
                )

                if cybz(
                    ddwz, ywz_pos, self.bz, self.YXBJ
                ):
                    zzc.add(round(t, 1))
                    break

    return len(zzc) * self.SJJG

def jssyd(self, gt):
    """计算适应度"""
    zcbsj = self.jszcbsj(gt)
    wf = self.jyyq(gt)

    if wf > 0:
        return max(0, zcbsj - 1000 * wf)

    return zcbsj

def jyyq(self, gt):
    """检验约束"""
    cs = gt.reshape(3, 4)
    wf = 0

    for c in cs:
        fx, sd, tcsj, qbyc = c

```

```

        if not (self.WRSD[0] <= sd <= self.WRSD[1]):
            wf += 1
        if not (0 <= fx <= 360):
            wf += 1
        if tcsj < 0:
            wf += 1
        if not (1.0 <= qbyc <= 15.0):
            wf += 1

    return wf

def xfgt(self, gt):
    """修复个体"""
    cs = gt.reshape(3, 4)
    xfc = []

    for i, c in enumerate(cs):
        fx, sd, tcsj, qbyc = c

        fx = np.clip(fx, 0, 360)
        sd = np.clip(sd, self.WRSD[0], self.WRSD[1])
        tcsj = max(0.5, tcsj)
        qbyc = np.clip(qbyc, 1.0, 15.0)

        wrid = ["FY1", "FY2", "FY3"][i]
        cs_wz = self.hdwrvz(wrid, tcsj, fx, sd)
        if cs_wz[1] < 0:
            wrwz = self.WRCWZ[wrid]
            if wrwz[1] < 0:
                fx = random.uniform(30, 150)
            else:
                fx = random.uniform(0, 180)

        xfc.append([fx, sd, tcsj, qbyc])

    jg = []
    for p in xfc:
        jg.extend(p)

    return np.array(jg)

def zsyby(self, gt, yc, zdyc):
    """自适应变异"""
    by = gt.copy()

    for i in range(12):
        if random.random() < self.BYL:
            cs1x = i % 4
            if cs1x == 0:
                by[i] = (by[i] + random.gauss(0, 30)) % 360
            elif cs1x == 1:
                by[i] += random.gauss(0, 15)
            elif cs1x == 2:
                by[i] += random.gauss(0, 3)
            else:
                by[i] += random.gauss(0, 2)

```



```

return self.xfgt(by)

def jcco(self, fq1, fq2):
    """交叉操作"""
    if random.random() > self.JCL:
        return fq1.copy(), fq2.copy()

    z1 = fq1.copy()
    z2 = fq2.copy()

    for i in range(3):
        if random.random() < 0.5:
            ks = i * 4
            js = ks + 4
            z1[ks:js] = fq2[ks:js]
            z2[ks:js] = fq1[ks:js]

    return self.xfgt(z1), self.xfgt(z2)

def jssxz(self, zj, syd, jss=5):
    """锦标赛选择"""
    jssy = random.sample(range(len(zj)), jss)
    jssyd = [syd[i] for i in jssy]
    yz = jssy[np.argmax(jssyd)]
    return zj[yz].copy()

def yh(self):
    print("修正版多无人机协同烟幕干扰策略优化...")
    print("修正内容:")
    print("1. 使用完整圆柱形目标采样点，提高遮蔽检测精度")
    print("2. 约束投放点避免Y负轴，确保能有效遮挡导弹")
    print(f"目标采样点数量: {len(self.bz)}")
    print("=" * 70)

    kssj = time.time()

    # 初始化种群
    zj = []
    for i in range(self.CLCS):
        if i < self.CLCS // 2:
            gt = self.scclgt()
        else:
            gt = np.random.uniform([0, 70, 1, 1] * 3, [360, 140, 25, 15] * 3)
        zj.append(self.xfgt(gt))

    zjsyd = -float('inf')
    zjgt = None
    wgsc = 0

    for yc in range(self.YCS):
        syd = [self.jssyd(ind) for ind in zj]
        wf = [self.jyyq(ind) for ind in zj]
        yxgt = sum(1 for v in wf if v == 0)

        pjsyd = np.mean(syd)
        zdsyd = np.max(syd)

```

```

self.sydlis.append(pjsyd)
self.zjsydlis.append(zdsyd)

zdyx = np.argmax(syd)
dqzj = syd[zdyx]

if dqzj > zjsyd:
    zjsyd = dqzj
    zjgt = zj[zdyx].copy()
    wgsc = 0

    print(f"第 {yc:3d} 代: 遮蔽时间={dqzj:.2f}s, "
          f"有效个体={yxgt}/{self.CLCS} ★ ")
else:
    wgsc += 1
    if yc % 20 == 0:
        print(f"第 {yc:3d} 代: 遮蔽时间={dqzj:.2f}s, "
              f"有效个体={yxgt}/{self.CLCS}")

if wgsc >= self.TZCS:
    print(f"\n连续{self.TZCS}代无改善，提前结束优化")
    break

xzj = []

jy = np.argsort(syd)[-self.JYS:]
for idx in jy:
    xzj.append(zj[idx].copy())

while len(xzj) < self.CLCS:
    fq1 = self.jssxz(zj, syd)
    fq2 = self.jssxz(zj, syd)

    z1, z2 = self.jcco(fq1, fq2)

    xzj.append(self.zsyby(z1, yc, self.YCS))
    if len(xzj) < self.CLCS:
        xzj.append(self.zsyby(z2, yc, self.YCS))

zj = xzj

yhsj = time.time() - kssj

if zjgt is not None:
    zzcbjsj = self.jszcbjsj(zjgt)
    print(f"\n优化完成!")
    print(f"最佳遮蔽时间: {zzcbjsj:.2f}s")
    print(f"优化耗时: {yhsj:.2f}秒")
else:
    print("\n优化失败，未找到有效解")

print("=" * 70)
return zjgt

def gsjsj(self, zjgt):
    """格式化结果"""

```

```

if zjgt is None:
    print("无有效解可显示")
    return None

cs = zjgt.reshape(3, 4)

print("\n=== 策略详情 ===")
zzcbsj = self.jszcbsj(zjgt)
print(f"总有效遮蔽时间: {zzcbsj:.2f}s")
print()

jg = {}
for i, wrid in enumerate(["FY1", "FY2", "FY3"]):
    fx, sd, tcsj, qbyc = cs[i]
    sdxl = self.hdwrsd(fx, sd)

    wrwz = self.hdwrrwz(wrid, tcsj, fx, sd)

    x_qb = wrwz[0] + sdxl[0] * qbyc
    y_qb = wrwz[1] + sdxl[1] * qbyc
    z_qb = wrwz[2] - 0.5 * self.ZL * qbyc **2
    qbw = np.array([x_qb, y_qb, z_qb])

    qbsj = tcsj + qbyc
    zzbj = qbsj + self.YXSY

    jg[wrid] = {
        "fx": fx,
        "sd": sd,
        "tcsj": tcsj,
        "qbyc": qbyc,
        "tcwz": wrwz,
        "qbw": qbw
    }

    print(f"{wrid} 策略:")
    print(f" 飞行方向: {fx:.1f}°")
    print(f" 飞行速度: {sd:.1f} m/s")
    print(f" 投放时间: {tcsj:.2f}s")
    print(f" 起爆延迟: {qbyc:.2f}s")
    print(f" 起爆时间: {qbsj:.2f}s → {zzbj:.2f}s")
    print(f" 投放点: ({wrwz[0]:.1f}, {wrwz[1]:.1f}, {wrwz[2]:.1f})")
    print(f" 起爆点: ({qbz[0]:.1f}, {qbz[1]:.1f}, {qbz[2]:.1f})")

    # 检查Y坐标
    if wrwz[1] < 0:
        print(f" 警告: 投放点Y坐标为负值, 可能无法有效遮挡")
        print()

return jg

def bcjgxls(self, zjgt, mc="result2_corrected.xlsx"):
    """保存结果到Excel"""
    if zjgt is None:
        print("无有效解可保存")

```

```

        return

    cs = zjgt.reshape(3, 4)

    sj = []
    for i, wrid in enumerate(["FY1", "FY2", "FY3"]):
        fx, sd, tcsj, qbyc = cs[i]
        sdxl = self.hdwrsd(fx, sd)

        wrwz = self.hdwrvz(wrid, tcsj, fx, sd)

        x_qb = wrwz[0] + sdxl[0] * qbyc
        y_qb = wrwz[1] + sdxl[1] * qbyc
        z_qb = wrwz[2] - 0.5 * self.ZL * qbyc** 2

        sj.append({
            '无人机编号': wrid,
            '飞行方向(度)': round(fx, 1),
            '飞行速度(m/s)': round(sd, 1),
            '投放时间(s)': round(tcsj, 2),
            '起爆延迟(s)': round(qbyc, 2),
            '投放点X(m)': round(wrwz[0], 1),
            '投放点Y(m)': round(wrwz[1], 1),
            '投放点Z(m)': round(wrwz[2], 1),
            '起爆点X(m)': round(x_qb, 1),
            '起爆点Y(m)': round(y_qb, 1),
            '起爆点Z(m)': round(z_qb, 1)
        })

    df = pd.DataFrame(sj)
    df.to_excel(mc, index=False)

    zzcbsj = self.jszcbsj(zjgt)
    print(f"\n结果已保存到 {mc}")
    print(f"总有效遮蔽时间: {zzcbsj:.2f}s")

def main():
    print("多无人机烟幕干扰协同策略优化系统")
    print("=" * 70)
    print("1. 目标模型: 使用完整圆柱形采样点而非简化中心点")
    print("2. 投放约束: 确保投放点不在Y负轴，避免无效遮挡")
    print("3. 策略生成: 基于导弹轨迹的智能初始化")
    print("=" * 70)

    # 设置随机种子
    random.seed(42)
    np.random.seed(42)

    yhq = XZDWRJYH()

    zjg = yhq.yh()

    if zjg is not None:
        wf = yhq.jyyq(zjg)

```

```

yx = (wf == 0)

print(f"\n解的可行性: {' 通过' if yx else ' 不通过'}")
if yx:
    jgsj = yhq.gsjg(zjjg)
    yhq.bcjgxls(zjjg, "result2_corrected.xlsx")
    print("\n优化完成! 结果已保存为Excel文件")
else:
    print("解不满足约束条件")
else:
    print("未找到有效解")

if __name__ == "__main__":
    main()

```

代码5：问题五求解

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import random
import time
import warnings
from itertools import combinations

warnings.filterwarnings('ignore')

plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

def jl_3d_ks(p1, p2):
    return np.sqrt((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2 + (p1[2] - p2[2]) ** 2)

def xd_qq_xj_ks(p1, p2, qq_zx, bj):
    xd_xl = p2 - p1
    p1_dao_zx = qq_zx - p1

    xd_cd_pf = np.dot(xd_xl, xd_xl)
    if xd_cd_pf == 0:
        return jl_3d_ks(p1, qq_zx) <= bj

    t = np.dot(p1_dao_zx, xd_xl) / xd_cd_pf

    if t < 0.0:
        zj_d = p1
    elif t > 1.0:
        zj_d = p2
    else:
        zj_d = p1 + t * xd_xl

    jl = jl_3d_ks(zj_d, qq_zx)
    return jl <= bj

```

```

def hq_ym_wz_ks(tp_wz, tp_sd, t_cong_tp, t_cong_bz, zl, ym_xc_sd):
    if t_cong_bz < 0:
        x = tp_wz[0] + tp_sd[0] * t_cong_tp
        y = tp_wz[1] + tp_sd[1] * t_cong_tp
        z = tp_wz[2] - 0.5 * zl * t_cong_tp * t_cong_tp
        return np.array([x, y, z])
    else:
        bz_sj = t_cong_tp - t_cong_bz
        x_bz = tp_wz[0] + tp_sd[0] * bz_sj
        y_bz = tp_wz[1] + tp_sd[1] * bz_sj
        z_bz = tp_wz[2] - 0.5 * zl * bz_sj * bz_sj

        x = x_bz
        y = y_bz
        z = z_bz - ym_xc_sd * t_cong_bz

    return np.array([x, y, z])

def jc_sy_yd_gsr_ks(dd_wz, ym_wz, mb_yld, yx_bj):
    if np.any(np.isnan(ym_wz)):
        return False

    for yl in mb_yld:
        if not xd_qq_xj_ks(dd_wz, yl, ym_wz, yx_bj):
            return False
    return True

class ZqDdDwjYmYhq:
    def __init__(self):
        self.DD_SD = 300
        self.YM_XC_SD = 3
        self.YX_BJ = 10
        self.YX_SJ = 20
        self.MB_BJ = 7
        self.MB_GD = 10
        self.WRJ_SD_FW = [70, 140]
        self.ZL = 9.8
        self.ZX_TP_JG = 1.0

        self.WRJ_CS = {
            "FY1": np.array([17800, 0, 1800]),
            "FY2": np.array([12000, 1400, 1400]),
            "FY3": np.array([6000, -3000, 700]),
            "FY4": np.array([11000, 2000, 1800]),
            "FY5": np.array([13000, -2000, 1300])
        }

        self.DD_CS = {
            "M1": np.array([20000, 0, 2000]),
            "M2": np.array([19000, 600, 2100]),
            "M3": np.array([18000, -600, 1900])
        }

```

```

self.JMB = np.array([0, 0, 0])
self.MB_ZX = np.array([0, 200, self.MB_GD / 2])

self.mb_yld = self.sc_yzt_mb_yld()

self.ys_dd_gj()
self.ys_lj_d()

self.ZQ_SIZE = 200
self.DAI_SHU = 400
self.BY_LV = 0.2
self.JC_LV = 0.8
self.JY_SIZE = 50
self.ZZ_TZ_NSX = 60

self.SJ_BZ = 0.1
self.ZD_SJ = 80

self.spd_ls = []
self.zj_spd_ls = []

def sc_yzt_mb_yld(self):
    yld = []
    zx_x, zx_y, zx_z = self.MB_ZX
    bj = self.MB_BJ
    gd = self.MB_GD

    yld.append(np.array([zx_x, zx_y, zx_z]))

    for z_py in [0, gd / 4, gd / 2, 3 * gd / 4, gd]:
        z = zx_z - gd / 2 + z_py

        for r_bl in [0.3, 0.6, 1.0]:
            r = bj * r_bl
            jd_ls = [0, np.pi / 4, np.pi / 2, 3 * np.pi / 4, np.pi, 5 * np.pi / 4, 3 * np.pi / 2, 7 * np.pi
/ 4]
            for jd in jd_ls:
                x = zx_x + r * np.cos(jd)
                y = zx_y + r * np.sin(jd)
                yld.append(np.array([x, y, z]))

    return np.array(yld)

def ys_dd_gj(self):
    self.dd_xx = {}
    for dd_id, cs_wz in self.DD_CS.items():
        fx = self.JMB - cs_wz
        jl = np.linalg.norm(fx)
        dx_fx = fx / jl if jl > 0 else np.array([0, 0, 0])
        dao_mb_sj = jl / self.DD_SD

        self.dd_xx[dd_id] = {
            'cs_wz': cs_wz,
            'dx_fx': dx_fx,
            'dao_mb_sj': dao_mb_sj
        }

```

```

print("导弹轨迹信息:")
for dd_id, xx in self.dd_xx.items():
    print(f"{dd_id}: 飞行时间 {xx['dao_mb_sj']:.1f}s")

def ys_lj_d(self):
    self.lj_d_ls = {}

    for dd_id in ["M1", "M2", "M3"]:
        d_ls = []
        dao_mb_sj = self.dd_xx[dd_id]['dao_mb_sj']

        gj_sj_ls = []
        gj_sj_ls.extend(np.linspace(dao_mb_sj * 0.2, dao_mb_sj * 0.4, 3))
        gj_sj_ls.extend(np.linspace(dao_mb_sj * 0.4, dao_mb_sj * 0.7, 4))
        gj_sj_ls.extend(np.linspace(dao_mb_sj * 0.7, dao_mb_sj * 0.9, 3))

        for t in gj_sj_ls:
            dd_wz = self.hq_dd_wz(dd_id, t)
            mb_fx = self.MB_ZX - dd_wz
            mb_jl = np.linalg.norm(mb_fx)
            if mb_jl > 0:
                for bl in [0.3, 0.5, 0.7]:
                    lj_wz = dd_wz + mb_fx * bl
                    d_ls.append((t, lj_wz))

        self.lj_d_ls[dd_id] = d_ls

    print(f"生成拦截点: {sum(len(d_ls) for d_ls in self.lj_d_ls.values())}个")

def hq_dd_wz(self, dd_id, t):
    xx = self.dd_xx[dd_id]
    return xx['cs_wz'] + xx['dx_fx'] * self.DD_SD * t

def hq_wrj_wz(self, wrj_id, t, fx_d, sd):
    fx_hd = np.radians(fx_d)
    dx_fx = np.array([np.cos(fx_hd), np.sin(fx_hd), 0])
    xin_wz = self.WRJ_CS[wrj_id] + dx_fx * sd * t
    xin_wz[2] = self.WRJ_CS[wrj_id][2]
    return xin_wz

def hq_wrj_sd_xl(self, fx_d, sd):
    fx_hd = np.radians(fx_d)
    return np.array([
        sd * np.cos(fx_hd),
        sd * np.sin(fx_hd),
        0
    ])

def sc_cl_gt(self):
    gt = []

    wrj_ids = ["FY1", "FY2", "FY3", "FY4", "FY5"]
    dd_ids = ["M1", "M2", "M3"]

    wrj_fp = {}
    for i, wrj_id in enumerate(wrj_ids):

```



```

if i < 3:
    wrj_fp[wrj_id] = dd_ids[i]
else:
    wrj_fp[wrj_id] = random.choice(dd_ids)

for wrj_id in wrj_ids:
    mb_dd = wrj_fp[wrj_id]
    wrj_wz = self.WRJ_CS[wrj_id]

    kyd_d_ls = self.lj_d_ls[mb_dd]
    if kyd_d_ls:
        ym_shu = random.randint(2, 3)
        xz_d_ls = random.sample(kyd_d_ls, min(ym_shu, len(kyd_d_ls)))
        xz_d_ls.sort(key=lambda x: x[0])
    else:
        ym_shu = random.randint(1, 3)
        xz_d_ls = []
        dao_mb_sj = self.dd_xx[mb_dd]['dao_mb_sj']
        for i in range(ym_shu):
            t = random.uniform(dao_mb_sj * 0.3, dao_mb_sj * 0.8)
            wz = self.hq_dd_wz(mb_dd, t)
            xz_d_ls.append((t, wz))

    if xz_d_ls:
        dy_mb = xz_d_ls[0][1]
        fx_xl = dy_mb - wrj_wz
        jc_fx = np.degrees(np.arctan2(fx_xl[1], fx_xl[0]))
        if jc_fx < 0:
            jc_fx += 360

        cs_wz = wrj_wz + fx_xl * 0.5
        if cs_wz[1] < 0 and wrj_wz[1] < 100:
            jc_fx = random.uniform(30, 150)
        else:
            jc_fx = random.uniform(0, 360)

        sd = random.uniform(100, 140)

        for i in range(3):
            if i < len(xz_d_ls):
                mb_sj, mb_wz = xz_d_ls[i]

                dao_mb_jl = np.linalg.norm(mb_wz[:2] - wrj_wz[:2])
                lx_sj = dao_mb_jl / sd
                tp_sj = max(1.0, mb_sj - lx_sj - random.uniform(3, 8))
                bz_yc = random.uniform(3, 8)
                hd = 1
            else:
                tp_sj = 0
                bz_yc = 0
                hd = 0

            gt.extend([jc_fx, sd, tp_sj, bz_yc, hd])

    return np.array(gt)

def js_fgd_sj_zq(self, gt):

```

```

cs = gt.reshape(5, 3, 5)

zh_fgd = {}
fgd_zl = {}

for dd_id in ["M1", "M2", "M3"]:
    zh_fgd[dd_id] = set()
    fgd_zl[dd_id] = 0

for t in np.arange(0, self.ZD_SJ, self.SJ_BZ):
    for dd_id in ["M1", "M2", "M3"]:
        if t > self.dd_xx[dd_id]['dao_mb_sj']:
            continue

        dd_wz = self.hq_dd_wz(dd_id, t)

        dao_mb_jl = np.linalg.norm(dd_wz[:2] - self.MB_ZX[:2])
        jl_qz = max(0.1, 1.0 - dao_mb_jl / 20000)

        fgd_yx = False
        fgd_js = 0

    for wrj_idx, wrj_id in enumerate(["FY1", "FY2", "FY3", "FY4", "FY5"]):
        wrj_cs = cs[wrj_idx]
        fx, sd = wrj_cs[0, 0], wrj_cs[0, 1]

        for ym_idx in range(3):
            ym_cs = wrj_cs[ym_idx]
            _, _, tp_sj, bz_yc, hd = ym_cs

            if hd == 0:
                continue

            t_cong_tp = t - tp_sj
            t_cong_bz = t - tp_sj - bz_yc

            if (t_cong_tp >= 0 and t_cong_bz >= 0 and
                t_cong_bz <= self.YX_SJ):

                wrj_wz = self.hq_wrj_wz(wrj_id, tp_sj, fx, sd)

                if wrj_wz[1] < -50:
                    jl_qz *= 0.3

                sd_xl = self.hq_wrj_sd_xl(fx, sd)
                ym_wz = hq_ym_wz_ks(
                    wrj_wz, sd_xl, t_cong_tp, t_cong_bz,
                    self.ZL, self.YM_XC_SD
                )

                if jc_sy_yd_gsr_ks(
                    dd_wz, ym_wz, self.mb_yld, self.YX_BJ
                ):
                    fgd_yx = True
                    fgd_js += 1

        if fgd_yx:

```

```

        zh_fgd[dd_id].add(round(t, 1))
        zl_jl = jl_qz * (1 + 0.1 * (fgd_js - 1))
        fgd_zl[dd_id] += zl_jl

    zh_sj = 0
    zh_zl = 0

    for dd_id in ["M1", "M2", "M3"]:
        dd_sj = len(zh_fgd[dd_id]) * self.SJ_BZ
        zh_sj += dd_sj
        zh_zl += fgd_zl[dd_id]

    ph_jl = 0
    fgd_dd_shu = sum(1 for mid in ["M1", "M2", "M3"]
                     if len(zh_fgd[mid]) > 0)
    if fgd_dd_shu == 3:
        ph_jl = 5.0
    elif fgd_dd_shu == 2:
        ph_jl = 2.0

    return zh_sj + zh_zl * 0.1 + ph_jl, zh_fgd

def js_spd(self, gt):
    fgd_df, _ = self.js_fgd_sj_zq(gt)
    wg_shu = self.jc_ys(gt)

    if wg_shu > 0:
        return max(0, fgd_df - 100 * wg_shu)

    return fgd_df

def jc_ys(self, gt):
    cs = gt.reshape(5, 3, 5)
    wg_shu = 0

    for wrj_idx, wrj_cs in enumerate(cs):
        fx, sd = wrj_cs[0, 0], wrj_cs[0, 1]

        if not (self.WRJ_SD_FW[0] <= sd <= self.WRJ_SD_FW[1]):
            wg_shu += 1
        if not (0 <= fx <= 360):
            wg_shu += 1

    tp_sj_ls = []
    for ym_idx in range(3):
        _, _, tp_sj, bz_yc, hd = wrj_cs[ym_idx]
        if hd > 0:
            if tp_sj < 0:
                wg_shu += 1
            if not (1.0 <= bz_yc <= 15.0):
                wg_shu += 1
            tp_sj_ls.append(tp_sj)

    if len(tp_sj_ls) > 1:
        tp_sj_ls.sort()
        for i in range(1, len(tp_sj_ls)):
            if tp_sj_ls[i] - tp_sj_ls[i - 1] < self.ZX_TP_JG:

```

```

        wg_shu += 1

    return wg_shu

def xf_gt(self, gt):
    cs = gt.reshape(5, 3, 5)
    xf_cs = []

    for wrj_idx, wrj_cs in enumerate(cs):
        fx, sd = wrj_cs[0, 0], wrj_cs[0, 1]

        fx = np.clip(fx, 0, 360)
        sd = np.clip(sd, self.WRJ_SD_FW[0], self.WRJ_SD_FW[1])

        wrj_id = ["FY1", "FY2", "FY3", "FY4", "FY5"][wrj_idx]
        cs_wz = self.hq_wrj_wz(wrj_id, 5.0, fx, sd)
        if cs_wz[1] < -100:
            wrj_wz = self.WRJ_CS[wrj_id]
            if wrj_wz[1] < 0:
                fx = random.uniform(30, 150)
            else:
                fx = random.uniform(-30, 30)

        ym_cs = []
        tp_sj_ls = []

        for ym_idx in range(3):
            _, _, tp_sj, bz_yc, hd = wrj_cs[ym_idx]

            if hd > 0.5:
                tp_sj = max(0.5, tp_sj)
                bz_yc = np.clip(bz_yc, 1.0, 15.0)
                tp_sj_ls.append(tp_sj)
                hd = 1
            else:
                tp_sj = 0
                bz_yc = 0
                hd = 0

            ym_cs.append([fx, sd, tp_sj, bz_yc, hd])

        if len(tp_sj_ls) > 1:
            tp_sj_ls.sort()
            hd_cs = [p for p in ym_cs if p[4] > 0]
            hd_cs.sort(key=lambda x: x[2])

            for i in range(1, len(hd_cs)):
                if hd_cs[i][2] - hd_cs[i - 1][2] < self.ZX_TP_JG:
                    hd_cs[i][2] = hd_cs[i - 1][2] + self.ZX_TP_JG

        xf_cs.extend([p for sublist in ym_cs for p in sublist])

    return np.array(xf_cs)

def zsy_by(self, gt, ds, zd_ds):
    by_gt = gt.copy()
    cs = by_gt.reshape(5, 3, 5)

```

```

jd = ds / zd_ds
dt_by_lv = self.BY_LV * (1.2 - jd)

for wrj_idx in range(5):
    for ym_idx in range(3):
        for cs_idx in range(5):
            if random.random() < dt_by_lv:
                cs_lx = cs_idx
                if cs_lx == 0:
                    cs[wrj_idx, ym_idx, cs_idx] = (
                        cs[wrj_idx, ym_idx, cs_idx] +
                        random.gauss(0, 45)
                    ) % 360
                elif cs_lx == 1:
                    cs[wrj_idx, ym_idx, cs_idx] += random.gauss(0, 20)
                elif cs_lx == 2:
                    cs[wrj_idx, ym_idx, cs_idx] += random.gauss(0, 5)
                elif cs_lx == 3:
                    cs[wrj_idx, ym_idx, cs_idx] += random.gauss(0, 3)
                elif cs_lx == 4:
                    if random.random() < 0.15:
                        cs[wrj_idx, ym_idx, cs_idx] = 1 - cs[wrj_idx, ym_idx, cs_idx]

return self.xf_gt(cs.flatten())

def jc(self, fq1, fq2):
    if random.random() > self.JC_LV:
        return fq1.copy(), fq2.copy()

    zd1 = fq1.copy()
    zd2 = fq2.copy()

    cs1 = zd1.reshape(5, 3, 5)
    cs2 = zd2.reshape(5, 3, 5)

    jc_lx = random.choice(['wrj', 'ym', 'hh'])

    if jc_lx == 'wrj':
        for wrj_idx in range(5):
            if random.random() < 0.5:
                lsh = cs1[wrj_idx].copy()
                cs1[wrj_idx] = cs2[wrj_idx]
                cs2[wrj_idx] = lsh
    elif jc_lx == 'ym':
        for wrj_idx in range(5):
            for ym_idx in range(3):
                if random.random() < 0.5:
                    lsh = cs1[wrj_idx, ym_idx].copy()
                    cs1[wrj_idx, ym_idx] = cs2[wrj_idx, ym_idx]
                    cs2[wrj_idx, ym_idx] = lsh
    else:
        jc_d = random.randint(1, 74)
        zd1_pt = cs1.flatten()
        zd2_pt = cs2.flatten()

        zd1_pt[jc_d:] = zd2_pt[jc_d:]

```

```

        zd2_pt[jc_d:] = cs1.flatten()[jc_d:]

        cs1 = zd1_pt.reshape(5, 3, 5)
        cs2 = zd2_pt.reshape(5, 3, 5)

    return self.xf_gt(cs1.flatten()), self.xf_gt(cs2.flatten())

def js_xz(self, zq, spd_fs, js_size=7):
    js_sy = random.sample(range(len(zq)), js_size)
    js_spd = [spd_fs[i] for i in js_sy]
    ys_sy = js_sy[np.argmax(js_spd)]
    return zq[ys_sy].copy()

def yhh(self):
    print("多导弹多无人机烟幕干扰策略优化...")
    print("目标: 3枚导弹 (M1, M2, M3)")
    print("资源: 5架无人机 (FY1-FY5), 每架最多3枚烟幕弹")
    print("智能拦截点 + 多重奖励 + 自适应算法")
    print("=" * 70)

    ks_sj = time.time()

    zq = []
    for i in range(self.ZQ_SIZE):
        if i < self.ZQ_SIZE * 0.7:
            gt = self.sc_cl_gt()
        else:
            gt = []
            for _ in range(5):
                fx = random.uniform(0, 360)
                sd = random.uniform(80, 140)
                for _ in range(3):
                    tp_sj = random.uniform(1, 40)
                    bz_yc = random.uniform(2, 12)
                    hd = random.choice([0, 1])
                    gt.extend([fx, sd, tp_sj, bz_yc, hd])
            gt = np.array(gt)

        zq.append(self.xf_gt(gt))

    zj_spd = -float("inf")
    zj_gt = None
    wy_gj_js = 0

    for ds in range(self.DAI_SHU):
        spd_fs = [self.js_spd(gt) for gt in zq]
        wg_shu_ls = [self.jc_ys(gt) for gt in zq]
        yx_gt_shu = sum(1 for v in wg_shu_ls if v == 0)

        pj_spd = np.mean(spd_fs)
        zd_spd = np.max(spd_fs)
        self.spd_ls.append(pj_spd)
        self.zj_spd_ls.append(zd_spd)

        zd_sy = np.argmax(spd_fs)
        dq_zj = spd_fs[zd_sy]

```

```

if dq_zj > zj_spd:
    zj_spd = dq_zj
    zj_gt = zq[zd_sy].copy()
    wy_gj_js = 0

    zs_fgd, _ = self.js_cc_fgd_sj(zj_gt)
    print(f"第 {ds:3d} 代: 遮蔽时间={zs_fgd:.2f}s, 适应度={dq_zj:.2f}, "
          f"有效个体={yx_gt_shu}/{self.ZQ_SIZE} ★ ")
else:
    wy_gj_js += 1
    if ds % 30 == 0:
        zs_fgd, _ = self.js_cc_fgd_sj(
            zj_gt) if zj_gt is not None else (0, {})
        print(f"第 {ds:3d} 代: 遮蔽时间={zs_fgd:.2f}s, 适应度={dq_zj:.2f}, "
              f"有效个体={yx_gt_shu}/{self.ZQ_SIZE}")

    if wy_gj_js >= self.ZZ_TZ_NSX:
        print(f"\n连续{self.ZZ_TZ_NSX}代无改善, 提前结束优化")
        break

xin_zq = []

jy_sy = np.argsort(spд_fs)[-self.JY_SIZE:]
for sy in jy_sy:
    xin_zq.append(zq[sy].copy())

while len(xin_zq) < self.ZQ_SIZE:
    fq1 = self.js_xz(zq, spд_fs)
    fq2 = self.js_xz(zq, spд_fs)

    zd1, zd2 = self.jc(fq1, fq2)

    xin_zq.append(self.zsy_by(zd1, ds, self.DAI_SHU))
    if len(xin_zq) < self.ZQ_SIZE:
        xin_zq.append(self.zsy_by(zd2, ds, self.DAI_SHU))

zq = xin_zq

yhh_sj = time.time() - ks_sj

if zj_gt is not None:
    zz_fgd, fgd_xq = self.js_cc_fgd_sj(zj_gt)
    print(f"\n优化完成!")
    print(f"最佳总遮蔽时间: {zz_fgd:.2f}s")
    for dd_id in ["M1", "M2", "M3"]:
        dd_fgd = len(fgd_xq[dd_id]) * self.SJ_BZ
        print(f" {dd_id}: {dd_fgd:.2f}s")
    print(f"优化耗时: {yhh_sj:.2f}秒")
else:
    print("\n优化失败, 未找到有效解")

print("=" * 70)
return zj_gt

def js_cc_fgd_sj(self, gt):
    cs = gt.reshape(5, 3, 5)

```

```

zh_fgd = {}
for dd_id in ["M1", "M2", "M3"]:
    zh_fgd[dd_id] = set()

for t in np.arange(0, self.ZD_SJ, self.SJ_BZ):
    for dd_id in ["M1", "M2", "M3"]:
        if t > self.dd_xx[dd_id]['dao_mb_sj']:
            continue

        dd_wz = self.hq_dd_wz(dd_id, t)

        for wrj_idx, wrj_id in enumerate(["FY1", "FY2", "FY3", "FY4", "FY5"]):
            wrj_cs = cs[wrj_idx]
            fx, sd = wrj_cs[0, 0], wrj_cs[0, 1]

            for ym_idx in range(3):
                ym_cs = wrj_cs[ym_idx]
                _, _, tp_sj, bz_yc, hd = ym_cs

                if hd == 0:
                    continue

                t_cong_tp = t - tp_sj
                t_cong_bz = t - tp_sj - bz_yc

                if (t_cong_tp >= 0 and t_cong_bz >= 0 and
                    t_cong_bz <= self.YX_SJ):

                    wrj_wz = self.hq_wrj_wz(wrj_id, tp_sj, fx, sd)
                    sd_xl = self.hq_wrj_sd_xl(fx, sd)
                    ym_wz = hq_ym_wz_ks(
                        wrj_wz, sd_xl, t_cong_tp, t_cong_bz,
                        self.ZL, self.YM_XC_SD
                    )

                    if jc_sy_yd_gsr_ks(
                        dd_wz, ym_wz, self.mb_yld, self.YX_BJ
                    ):
                        zh_fgd[dd_id].add(round(t, 1))
                        break
            else:
                continue
        break

    zh_sj = 0
    for dd_id in ["M1", "M2", "M3"]:
        zh_sj += len(zh_fgd[dd_id]) * self.SJ_BZ

    return zh_sj, zh_fgd

def gsf_jg(self, zj_gt):
    if zj_gt is None:
        print("无有效解可显示")
        return None

    cs = zj_gt.reshape(5, 3, 5)

```



```

zh_fgd, fgd_xq = self.js_cc_fgd_sj(zj_gt)

print("\n=== 多导弹多无人机策略详情 ===")
print(f"总有效遮蔽时间: {zh_fgd:.2f}s")
for dd_id in ["M1", "M2", "M3"]:
    dd_fgd = len(fgd_xq[dd_id]) * self.SJ_BZ
    print(f"{dd_id} 遮蔽时间: {dd_fgd:.2f}s")
print()

jg_ls = []
ym_js = 0

for wrj_idx, wrj_id in enumerate(["FY1", "FY2", "FY3", "FY4", "FY5"]):
    wrj_cs = cs[wrj_idx]
    fx, sd = wrj_cs[0, 0], wrj_cs[0, 1]

    print(f"{wrj_id} 策略:")
    print(f" 飞行方向: {fx:.1f}°")
    print(f" 飞行速度: {sd:.1f} m/s")

    hd_ym = 0
    for ym_idx in range(3):
        _, _, tp_sj, bz_yc, hd = wrj_cs[ym_idx]

        if hd > 0.5:
            hd_ym += 1
            ym_js += 1

        wrj_wz = self.hq_wrj_wz(wrj_id, tp_sj, fx, sd)
        sd_xl = self.hq_wrj_sd_xl(fx, sd)

        x_bz = wrj_wz[0] + sd_xl[0] * bz_yc
        y_bz = wrj_wz[1] + sd_xl[1] * bz_yc
        z_bz = wrj_wz[2] - 0.5 * self.ZL * bz_yc ** 2

        bz_sj = tp_sj + bz_yc
        fgd_js = bz_sj + self.YX_SJ

        print(f" 烟幕弹{ym_idx + 1}:")
        print(f" 投放时间: {tp_sj:.2f}s")
        print(f" 起爆延迟: {bz_yc:.2f}s")
        print(f" 起爆时间: {bz_sj:.2f}s → {fgd_js:.2f}s")
        print(f" 投放点: ({wrj_wz[0]:.1f}, {wrj_wz[1]:.1f}, {wrj_wz[2]:.1f})")
        print(f" 起爆点: ({x_bz:.1f}, {y_bz:.1f}, {z_bz:.1f})")

    if wrj_wz[1] < 0:
        print(f" 警告: 投放点Y坐标为负值")

    jg_ls.append({
        'wrj_bh': wrj_id,
        'ym_bh': ym_js,
        'fx_fx_d': round(fx, 1),
        'fx_sd_ms': round(sd, 1),
        'tp_sj_s': round(tp_sj, 2),
        'bz_yc_s': round(bz_yc, 2),

```

```

        'tp_d_x_m': round(wrj_wz[0], 1),
        'tp_d_y_m': round(wrj_wz[1], 1),
        'tp_d_z_m': round(wrj_wz[2], 1),
        'bz_d_x_m': round(x_bz, 1),
        'bz_d_y_m': round(y_bz, 1),
        'bz_d_z_m': round(z_bz, 1)
    })

    print(f" 活跃烟幕弹数量: {hd_ym}/3")
    print()

    print(f"总计使用烟幕弹: {ym_js}枚")
    return jg_ls

def bc_jg_zhi_excle(self, zj_gt, wjm="jg3.xlsx"):
    if zj_gt is None:
        print("无有效解可保存")
        return

    jg_ls = self.gsf_jg(zj_gt)
    if not jg_ls:
        print("无结果可保存")
        return

    df = pd.DataFrame(jg_ls)
    df.to_excel(wjm, index=False)

    zh_fgd, fgd_xq = self.js_cc_fgd_sj(zj_gt)
    print(f"\n结果已保存到 {wjm}")
    print(f"总有效遮蔽时间: {zh_fgd:.2f}s")
    for dd_id in ["M1", "M2", "M3"]:
        dd_fgd = len(fgd_xq[dd_id]) * self.SJ_BZ
        print(f"{dd_id} 遮蔽时间: {dd_fgd:.2f}s")

def fx_fgd_sjx(self, zj_gt):
    if zj_gt is None:
        return

    print("\n=== 遮蔽时间线分析 ===")
    _, fgd_xq = self.js_cc_fgd_sj(zj_gt)

    for dd_id in ["M1", "M2", "M3"]:
        fgd_sj_ls = sorted(fgd_xq[dd_id])
        if fgd_sj_ls:
            print(f"{dd_id} 遮蔽时间段:")

            duan_ls = []
            if fgd_sj_ls:
                ks = fgd_sj_ls[0]
                js = fgd_sj_ls[0]

                for i in range(1, len(fgd_sj_ls)):
                    if fgd_sj_ls[i] - fgd_sj_ls[i - 1] <= self.SJ_BZ + 0.01:
                        js = fgd_sj_ls[i]
                    else:
                        duan_ls.append((ks, js))

```

```

        ks = fgd_sj_ls[i]
        js = fgd_sj_ls[i]
        duan_ls.append((ks, js))

    for i, (ks, js) in enumerate(duan_ls):
        cc = js - ks + self.SJ_BZ
        print(f" 段{i + 1}: {ks:.1f}s - {js:.1f}s (持续{cc:.1f}s)")

    zh_sj = len(fgd_sj_ls) * self.SJ_BZ
    print(f" 总遮蔽时间: {zh_sj:.2f}s")
else:
    print(f"{dd_id}: 无遮蔽")
print()

def sc_xq_bg(self, zj_gt):
    if zj_gt is None:
        return

    print("\n" + "=" * 80)
    print("问题五 详细分析报告".center(80))
    print("=" * 80)

    zh_fgd, fgd_xq = self.js_cc_fgd_sj(zj_gt)
    cs = zj_gt.reshape(5, 3, 5)

    print("\n1. 总体性能指标")
    print(f" 总遮蔽时间: {zh_fgd:.2f}s")
    print(f" 平均每导弹遮蔽时间: {zh_fgd / 3:.2f}s")

    print("\n2. 各导弹遮蔽详情")
    for dd_id in ["M1", "M2", "M3"]:
        dd_fgd = len(fgd_xq[dd_id]) * self.SJ_BZ
        fgd_bl = dd_fgd / self.dd_xx[dd_id]['dao_mb_sj'] * 100
        print(f" {dd_id}:")
        print(f"   遮蔽时间: {dd_fgd:.2f}s")
        print(f"   飞行总时间: {self.dd_xx[dd_id]['dao_mb_sj']:.2f}s")
        print(f"   遮蔽覆盖率: {fgd_bl:.1f}%")

    print("\n3. 资源利用情况")
    zh_ym = 0
    for wrj_idx, wrj_id in enumerate(["FY1", "FY2", "FY3", "FY4", "FY5"]):
        wrj_cs = cs[wrj_idx]
        hd_ym = sum(1 for i in range(3) if wrj_cs[i, 4] > 0.5)
        zh_ym += hd_ym
        print(f" {wrj_id}: {hd_ym}/3 枚烟幕弹")

    print(f" 总计: {zh_ym}/15 枚烟幕弹 (利用率: {zh_ym / 15 * 100:.1f}%)")

    print("\n4. 策略有效性分析")
    y_fz_js = 0
    yx_bs_js = 0

    for wrj_idx, wrj_id in enumerate(["FY1", "FY2", "FY3", "FY4", "FY5"]):
        wrj_cs = cs[wrj_idx]
        fx, sd = wrj_cs[0, 0], wrj_cs[0, 1]

```

```

        for ym_idx in range(3):
            if wrj_cs[ym_idx, 4] > 0.5:
                tp_sj = wrj_cs[ym_idx, 2]
                wrj_wz = self.hq_wrj_wz(wrj_id, tp_sj, fx, sd)
                if wrj_wz[1] < 0:
                    y_fz_js += 1
                if wrj_wz[1] > 0 and wrj_wz[0] < 15000:
                    yx_bs_js += 1

        print(f" Y坐标为负的投放点: {y_fz_js}个")
        print(f" 有效部署区域的投放点: {yx_bs_js}个")
        print(
            f" 策略可行性: {'优秀' if y_fz_js == 0 and yx_bs_js > zh_ym * 0.7 else '良好' if
            y_fz_js <= 2 else '需要改进'}")

        print("\n" + "=" * 80)

def zhu():
    print("问题五：多导弹多无人机烟幕干扰策略优化系统")
    print("=" * 70)
    print("问题描述:")
    print("- 3枚来袭导弹: M1, M2, M3")
    print("- 5架无人机: FY1, FY2, FY3, FY4, FY5")
    print("- 每架无人机最多投放3枚烟幕弹")
    print("- 目标：最大化对所有导弹的总遮蔽时间")
    print("\n增强特性:")
    print("- 智能拦截点预计算")
    print("- 多重质量奖励机制")
    print("- 平衡遮蔽奖励")
    print("- 自适应变异算法")
    print("=" * 70)

    random.seed(42)
    np.random.seed(42)

    yhq = ZqDdDwjYmYhq()

    print("\n开始优化...")
    ks_sj = time.time()
    zj_jj = yhq.yhh()
    yhh_sj = time.time() - ks_sj

    if zj_jj is not None:
        wg_shu = yhq.jc_ys(zj_jj)
        yx = (wg_shu == 0)

        print(f"\n解的可行性: {'通过' if yx else '不通过'}")
        if yx:
            print("\n正在生成结果...")
            jg_sj = yhq.gsf_jg(zj_jj)
            yhq.bc_jg_zhi_excle(zj_jj, "jg3.xlsx")

            yhq.fx_fgd_sjx(zj_jj)
            yhq.sc_xq_bg(zj_jj)

```

```
print(f"\n问题五增强版优化完成!")
print(f" 总耗时: {yhh_sj:.2f}秒")
print(f" 结果已保存为: jg3.xlsx")

else:
    print(f" 解不满足约束条件, 违规数量: {wg_shu}")
    print("建议调整算法参数重新优化")
else:
    print("未找到有效解")
    print("建议:")
    print("1. 检查拦截点设置")
    print("2. 调整奖励机制权重")
    print("3. 增加迭代次数")

print("\n" + "=" * 70)
print("程序执行完成")

if __name__ == "__main__":
    zhu()
```